

Loong密钥封装算法

陶成东

2026 年 6 月 27 日

目录

1	前言	3
2	算法描述	7
2.1	数学基础	7
2.2	Loong公钥加密算法	17
2.2.1	公钥和私钥	17
2.2.2	加密	18
2.2.3	解密	20
2.3	Loong密钥封装算法	22
2.4	参数	23
3	设计原理	24
3.1	半结构化的LWE问题	25
3.2	噪声分布	26
4	性能分析	26
4.1	性能	26
4.2	测试值	27
5	安全分析	28
5.1	安全证明	28
5.2	BKZ算法的复杂度	29
5.3	Loong密钥封装算法的实际安全评估	30
5.3.1	Primal Attack	30
5.3.2	Dual Attack	30
5.3.3	Loong密钥封装算法的攻击复杂度	31
6	优势和劣势	31

1 前言

本文介绍Loong密钥封装算法的基本原理和实现. Loong密钥封装算法是一族基于格理论的密码算法, 它的安全性基于半结构化的LWE问题(简称为SLWE问题)的困难性. 基于格理论的密码算法的早期研究可以追溯到上个世纪90年代. 1996年, Ajtai证明了小整数解问题(Small Integer Solution Problem, 简称SIS问题)等格问题的困难性在一般情况下和最坏情况下是等价的, 这一结论说明格问题十分适合用来设计公钥密码算法[1]. 许多公钥密码算法所基于的数学问题并没有这样的等价证明, 例如大整数分解问题、离散对数问题、随机线性码的解码问题和多变量多项式方程组的求解问题等. 在Ajtai的论文中, 他还设计了基于SIS问题的哈希函数, 从此开启了格密码学研究的大门.

1996年, Jeffrey Hoffstein, Jill Pipher, 和Joseph H. Silverman等三人共同提出了NTRU公钥加密算法[2]. NTRU公钥加密算法可视为基于格理论的密码算法, 它的加密和解密速度都很快, 可以应用到实际环境中, 但是原始的NTRU加密算法没有安全性证明, 它的安全性评估依赖于已知攻击方法的破解能力.

1997年, Ajtai构造了一个基于唯一最短向量问题(unique Shortest Vector Problem, 简称uSVP问题, 它是一个格的困难问题)的公钥加密算法[3]. 该算法的安全性可以规约到uSVP问题的困难性, 这意味着如果敌手能够在多项式时间内破解该加密算法, 则敌手就能够在多项式时间内求解uSVP问题. 可惜该加密算法的加密和解密速度太慢, 我们很难将它应用到现实环境中.

2005年, Regav提出了容错学习问题(Learning With Error Problem, 简称LWE问题), 并构造了基于LWE问题的公钥加密方案(简称Regav方案), 这是格密码学研究的重大的突破[4]. Regav方案的公钥和密文长度都比较长, 均为 $\mathcal{O}(n^2)$, 其中 n 为公钥矩阵的维数. 为了减少公钥和密文的长度, 2010年, Lyubashevsky, Peikert 和Regev 提出了RLWE问题, 并构造了基于RLWE问题的公钥加密算法[5]. 基于RLWE问题的公钥和密文长度为 $\mathcal{O}(n)$. 为了进一步减少密文的长度, 文献[6]本质上提出了一种密文压缩技术, 并利用这种技术设计了基于格的密钥交换协议. 文献[7, 8]扩展了密文压缩技术, 并将这种技术用于基于RLWE问题的加密方案中, 进一步减少了密文的大小.

由于RLWE问题的困难性依赖于特定环的代数结构, 尽管现在我们还不知道如何利用代数结构来有效地攻击基于RLWE问题的密码系统, 但是

它们存在被代数攻击的风险. 因此, 在2015年, 文献[9]提出了MLWE问题. MLWE问题保留了部分RLWE问题的代数结构, 以减少公钥和密文的长度, 提升加密和解密的速度, 同时避免了RLWE问题的强代数假设. MLWE问题同时兼顾安全和效率, 十分适合用于构造密码算法. 然而, 在实际应用的基于MLWE问题的密码算法中, 它们仍然具有较强的代数结构, 例如, Kyber密钥封装算法[10]和Dilithium签名算法[11].

为了减弱代数结构对密码安全的影响, 本文提出一种基于半结构化LWE问题的密钥封装算法, 我们称之为Loong密钥封装算法. Loong密钥封装算法是一族IND-CCA2安全的密码算法. 我们首先构造CPA-安全的公钥加密算法, 然后再利用Fujisaki-Okamoto变换构造IND-CCA2安全的密钥封装算法.

为了便于理解, 我们在此简要地描述CPA-安全的Loong公钥加密算法. 为了简便起见, 我们省略了公钥和密文的压缩和解压缩步骤. 在原始的基于LWE问题的加密算法中, 公钥通常是没有任何代数结构的普通矩阵. 在基于MLWE和RLWE问题的加密算法中, 公钥通常是具有代数结构的矩阵, 这些代数结构通常同构于某个多项式环, 例如 $\mathbb{Z}_q[X]/X^N + 1$ 或者 $\mathbb{Z}_q[X]/X^N - 1$. 在Loong公钥加密算法中, 公钥矩阵 $(\mathbf{A}, \mathbf{B})$ 是半结构化的矩阵, 具体地,

$$\mathbf{A} = \begin{pmatrix} \mathbf{A}_1 & \mathbf{A}_2 \\ \mathbf{A}_3 & \mathbf{A}_4 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} \mathbf{B}_1 \\ \mathbf{B}_2 \end{pmatrix},$$

其中矩阵 $\mathbf{A}_1, \mathbf{A}_2, \mathbf{A}_3, \mathbf{B}_1$ 是分块反循环矩阵, 每一个分块可以看成多项式环 $\mathbb{Z}_q[X]/X^N + 1$ 的多项式. 矩阵 $\mathbf{A}_4$ 和 $\mathbf{B}_2$ 是没有代数结构的普通矩阵.

设 N, k_1 和 k_2 为正整数, q 为素数, 且 $q > 2$. Loong公钥加密算法的公钥和私钥生成步骤如下:

1. 随机生成矩阵 $\mathbf{A} = \begin{pmatrix} \mathbf{A}_1 & \mathbf{A}_2 \\ \mathbf{A}_3 & \mathbf{A}_4 \end{pmatrix}$, 其中子矩阵 $\mathbf{A}_1 \in \mathbb{Z}_q^{k_1 N \times k_1 N}, \mathbf{A}_2 \in \mathbb{Z}_q^{k_1 N \times k_2 N}, \mathbf{A}_3 \in \mathbb{Z}_q^{k_2 N \times k_1 N}$ 是分块反循环矩阵, 每一个分块都是反循环矩阵. $\mathbf{A}_4 \in \mathbb{Z}_q^{k_2 N \times k_2 N}$ 是没有代数结构的普通矩阵.
2. 根矩中心二项分布 ψ_η 生成噪声矩阵 $\begin{pmatrix} \mathbf{S}_1 \\ \mathbf{S}_2 \end{pmatrix}$ 和 $\begin{pmatrix} \mathbf{E}_1 \\ \mathbf{E}_2 \end{pmatrix}$, 其中子矩阵 $\mathbf{S}_1$ 和 $\mathbf{E}_1$ 是 $k_1 N$ 行 N 列的分块反循环矩阵, $\mathbf{S}_2$ 是 $k_2 N$ 行 N 列的分块反循环矩阵. $\mathbf{E}_2$ 是 $k_2 N$ 行 N 列的没有代数结构的矩阵.

3. 计算 $\mathbf{B} = \begin{pmatrix} \mathbf{B}_1 \\ \mathbf{B}_2 \end{pmatrix} = \begin{pmatrix} \mathbf{A}_1 & \mathbf{A}_2 \\ \mathbf{A}_3 & \mathbf{A}_4 \end{pmatrix} \begin{pmatrix} \mathbf{S}_1 \\ \mathbf{S}_2 \end{pmatrix} + \begin{pmatrix} \mathbf{E}_1 \\ \mathbf{E}_2 \end{pmatrix}$, 其中 $\mathbf{B}_1 \in \mathbb{Z}_q^{k_1 N \times N}$ 是分块反循环矩阵, $\mathbf{B}_2 \in \mathbb{Z}_q^{k_2 N \times N}$ 是没有代数结构的普通矩阵.
4. 公钥为 $\begin{pmatrix} \mathbf{A}_1 & \mathbf{A}_2 \\ \mathbf{A}_3 & \mathbf{A}_4 \end{pmatrix}$ 和 $\begin{pmatrix} \mathbf{B}_1 \\ \mathbf{B}_2 \end{pmatrix}$, 私钥为 $\begin{pmatrix} \mathbf{S}_1 \\ \mathbf{S}_2 \end{pmatrix}$. 矩阵 $\mathbf{A}$ 可以利用种子比特生成, 因此我们只需要存储种子比特. 矩阵 $\mathbf{B}_1$ 是分块反循环矩阵, 因此我们只需要存储每个分块的第1行元素. 矩阵 $\mathbf{B}_2$ 是普通矩阵, 因此我们需要存储 $\mathbf{B}_2$ 的所有元素.

设 $\mathbf{m} \in \{0, 1\}^{N^2}$ 是明文比特, 我们首先将明文编码成 N 行 N 列的二元矩阵 $\mathbf{M}$. 设 $\Delta = 2^{-1} \bmod q$, Loong 公钥加密算法的加密步骤如下:

1. 根据中心二项分布, 生成噪声矩阵 $\begin{pmatrix} \mathbf{R}_1 & \mathbf{R}_2 \end{pmatrix}$, $\begin{pmatrix} \mathbf{E}_3 & \mathbf{E}_4 \end{pmatrix}$ 和 $\mathbf{E}_5$, 其中 $\mathbf{R}_1$ 是 N 行 $k_1 N$ 列的分块反循环矩阵, $\mathbf{R}_2$ 是 N 行 $k_2 N$ 列的分块反循环矩阵, $\mathbf{E}_3$ 是 N 行 $k_1 N$ 列的分块反循环矩阵, $\mathbf{E}_4$ 是 N 行 $k_2 N$ 列的没有代数结构的普通矩阵, $\mathbf{E}_5$ 是 N 行 N 列的没有代数结构的普通矩阵.
2. 计算 $\begin{pmatrix} \mathbf{C}_1 & \mathbf{C}_2 \end{pmatrix} = \begin{pmatrix} \mathbf{R}_1 & \mathbf{R}_2 \end{pmatrix} \begin{pmatrix} \mathbf{A}_1 & \mathbf{A}_2 \\ \mathbf{A}_3 & \mathbf{A}_4 \end{pmatrix} + \begin{pmatrix} \mathbf{E}_3 & \mathbf{E}_4 \end{pmatrix}$, 其中 $\mathbf{C}_1 \in \mathbb{Z}_q^{N \times k_1 N}$ 是分块反循环矩阵, $\mathbf{C}_2 \in \mathbb{Z}_q^{N \times k_2 N}$ 是无代数结构的普通矩阵.
3. 计算 $\mathbf{C}_3 = \begin{pmatrix} \mathbf{R}_1 & \mathbf{R}_2 \end{pmatrix} \begin{pmatrix} \mathbf{B}_1 \\ \mathbf{B}_2 \end{pmatrix} + \mathbf{E}_5 + \Delta \cdot \mathbf{M}$, 其中 $\mathbf{C}_3 \in \mathbb{Z}_q^{N \times N}$ 是没有代数结构的普通矩阵.
4. 密文为 $\mathbf{C}_1, \mathbf{C}_2, \mathbf{C}_3$. 因为 $\mathbf{C}_1$ 是分块循环矩阵, 因此我们只需存储每个分块的第1行元素, $\mathbf{C}_2$ 和 $\mathbf{C}_3$ 是普通矩阵, 需要存储所有元素. 我们总共需要存储 $k_1 N + k_2 N^2 + N^2$ 个元素.

Loong 公钥加密算法的解密步骤如下:

1. 计算 $\mathbf{U} = 2(\mathbf{C}_3 - \mathbf{C}_1 \mathbf{S}_1 - \mathbf{C}_2 \mathbf{S}_2) \bmod \pm q$.
2. 把矩阵 $\mathbf{U}$ 看成整数矩阵, 计算 $\mathbf{M} = \mathbf{U} \bmod 2$, $\mathbf{M}$ 就是明文.

在以往的基于LWE问题的加密算法中, 解密算法的最后一步通常是比较密文与 $q/4$ 的大小, 如果大于 $q/4$, 则对应的明文比特为1, 否则对应的明文比

特为0. 在Loong加密算法中, 我们只需要做模运算, 就可以恢复明文. 这与NTRU加密算法的解密过程相似, 提升了解密的速度.

我们将Loong公钥加密算法归纳总结为表1.

本文剩余部分结构如下: 第2节将详细描述Loong密钥封装算法和参数; 第3节介绍Loong 密钥封装算法的设计原理; 第4节介绍Loong密钥封装算法的实现和性能; 第5节介绍Loong密钥封装算法的安全性; 第6节介绍Loong密钥封装算法的优势和劣势.

KeyGen:

$$\begin{aligned}
1: & \mathbf{A} = \begin{pmatrix} \mathbf{A}_1 & \mathbf{A}_2 \\ \mathbf{A}_3 & \mathbf{A}_4 \end{pmatrix} \leftarrow \mathbb{Z}_q^{kN \times kN}, \begin{pmatrix} \mathbf{S}_1 \\ \mathbf{S}_2 \end{pmatrix} \leftarrow \psi_\eta, \begin{pmatrix} \mathbf{E}_1 \\ \mathbf{E}_2 \end{pmatrix} \leftarrow \psi_\eta \\
2: & \mathbf{B} = \begin{pmatrix} \mathbf{B}_1 \\ \mathbf{B}_2 \end{pmatrix} \leftarrow \begin{pmatrix} \mathbf{A}_1 & \mathbf{A}_2 \\ \mathbf{A}_3 & \mathbf{A}_4 \end{pmatrix} \begin{pmatrix} \mathbf{S}_1 \\ \mathbf{S}_2 \end{pmatrix} + \begin{pmatrix} \mathbf{E}_1 \\ \mathbf{E}_2 \end{pmatrix} \\
3: & \text{return } \text{pk} = (\mathbf{A}, \mathbf{B}), \text{sk} = \begin{pmatrix} \mathbf{S}_1 \\ \mathbf{S}_2 \end{pmatrix}
\end{aligned}$$

Encrypt(pk, M):

$$\begin{aligned}
1: & \begin{pmatrix} \mathbf{R}_1 & \mathbf{R}_2 \end{pmatrix} \leftarrow \psi_\eta, \begin{pmatrix} \mathbf{E}_3 & \mathbf{E}_4 \end{pmatrix} \leftarrow \psi_\eta, \mathbf{E}_5 \leftarrow \psi_\eta \\
2: & \begin{pmatrix} \mathbf{C}_1 & \mathbf{C}_2 \end{pmatrix} \leftarrow \begin{pmatrix} \mathbf{R}_1 & \mathbf{R}_2 \end{pmatrix} \begin{pmatrix} \mathbf{A}_1 & \mathbf{A}_2 \\ \mathbf{A}_3 & \mathbf{A}_4 \end{pmatrix} + \begin{pmatrix} \mathbf{E}_3 & \mathbf{E}_4 \end{pmatrix} \\
3: & \mathbf{C}_3 \leftarrow \begin{pmatrix} \mathbf{R}_1 & \mathbf{R}_2 \end{pmatrix} \begin{pmatrix} \mathbf{B}_1 \\ \mathbf{B}_2 \end{pmatrix} + \mathbf{E}_5 + \Delta \cdot \mathbf{M} \\
4: & \text{return } (\mathbf{C}_1, \mathbf{C}_2, \mathbf{C}_3)
\end{aligned}$$

Decrypt(sk, C₁, C₂, C₃):

$$\begin{aligned}
1: & \mathbf{M} \leftarrow (2(\mathbf{C}_3 - \mathbf{C}_1 \mathbf{S}_1 - \mathbf{C}_2 \mathbf{S}_2) \bmod \pm q) \bmod 2 \\
2: & \text{return } \mathbf{M}
\end{aligned}$$

表 1: Loong加密算法

2 算法描述

2.1 数学基础

整数模运算 设 $\mathbb{Z}$ 为整数环, q 为素数, $\mathbb{Z}_q$ 表示商环 $\mathbb{Z}/q\mathbb{Z}$. 设 a 为整数, 模运算 $r = a \bmod q$ 表示 $r \in \{0, 1, \dots, q-1\}$, 模运算 $r = a \bmod^{\pm} q$ 表示 $r \in \{-\frac{q-1}{2}, \dots, -1, 0, 1, \dots, \frac{q-1}{2}\}$. 记 $d_q = \lceil \log_2(q) \rceil$.

范数 设 $a \in \mathbb{Z}_q$, $\mathbf{a} = (a_0, a_1, \dots, a_{n-1}) \in \mathbb{Z}_q^n$, 我们定义

$$\|a\|_{\infty} = |a \bmod^{\pm} q|, \quad \|\mathbf{a}\|_{\infty} = \max_{0 \leq i \leq n-1} \|a_i\|_{\infty}.$$

$$\|\mathbf{a}\|_2 = (\|a_0\|_{\infty}^2 + \dots + \|a_{n-1}\|_{\infty}^2)^{1/2}.$$

压缩函数 设 d 为正偶数, $x \in \{0, 1, \dots, q-1\}$, 我们定义压缩函数如下:

$$\text{Compress}(x, d) = \left\lfloor \frac{x}{d} \right\rfloor.$$

记 $y = \text{Compress}(x, d)$, 我们定义解压函数如下:

$$\text{Decompress}(y, d) = \min \left(y \cdot d + \frac{d}{2}, q-1 \right).$$

当 d 为2的方幂时, 我们只需利用移位运算和加法运算就可以实现 $\mathbb{Z}_q$ 的元素的压缩和解压缩, 而不需要除法运算和模运算.

记 $\tilde{x} = \text{Decompress}(\text{Compress}(x, d), d)$, 对任意 $x \in \mathbb{Z}_q$, 容易证明 $|x - \tilde{x}| \leq \frac{d}{2}$. 事实上, 根据取整运算的定义, 存在整数 $r \in [0, d)$, 使得 $x = yd + r$. 我们接下来分两种情况证明. 若 $yd + \frac{d}{2} \leq q-1$, 则 $|x - \tilde{x}| = |r - \frac{d}{2}| \leq \frac{d}{2}$. 若 $yd + \frac{d}{2} > q-1$, 则 $\tilde{x} = q-1$. 另一方面 $x \geq yd > (q-1) - \frac{d}{2}$ (否则 $yd + \frac{d}{2} \leq q-1$). 所以 $|x - \tilde{x}| = (q-1) - x < \frac{d}{2}$.

设 $\mathbf{a} \in \mathbb{Z}_q^n$, $\mathbf{A} \in \mathbb{Z}_q^{n \times n}$, $\text{Compress}(\mathbf{a}, d)$ 和 $\text{Compress}(\mathbf{A}, d)$ 分别表示对向量 $\mathbf{a}$ 和矩阵 $\mathbf{A}$ 的每个元素做压缩运算. $\text{Decompress}(\mathbf{a}, d)$ 和 $\text{Decompress}(\mathbf{A}, d)$ 分别表示对向量 $\mathbf{a}$ 和矩阵 $\mathbf{A}$ 的每个元素做解压缩运算.

比特串与字节数组 设集合 $\mathbb{B} = \{0, 1, 2, \dots, 255\}$, t 为正整数. 算法1的功能是将比特数组 $\mathbf{b}$ 转换为字节数组 $\mathbf{bytes}$. 设比特串为 $b_0b_1b_2b_3b_4b_5b_6b_7$, 则对应的字节为 $b_0 + b_1 \cdot 2 + b_2 \cdot 2^2 + \dots + b_7 \cdot 2^7$, 即我们采用小端位序(LSB first order). 例如, 比特串11111011对应的字节为 $2^0 + 2^1 + 2^2 + 2^3 + 2^4 + 2^6 + 2^7 = 223$.

算法 1 BitsToBytes(**b**):比特数组转换为字节数组.

输入: 比特数组 $\mathbf{b} \in \{0, 1\}^{8t}$.

输出: 字节数组 $\mathbf{bytes} \in \mathbb{B}^t$.

```
1: bytes  $\leftarrow (0, \dots, 0)$ 
2: for ( $i = 0; i < t; i++$ ) do
3:    $\mathbf{bytes}[i] \leftarrow \sum_{j=0}^7 \mathbf{b}[8i + j] \cdot 2^j$ 
4: end for
5: return bytes
```

算法2的功能是将字节数组 $\mathbf{bytes}$ 转换为比特数组 $\mathbf{b}$. 设 $\mathbf{bytes}$ 的某个元素为 $b_0 + b_1 \cdot 2 + \dots + b_7 \cdot 2^7$, 则它对应的比特串为 $b_0 b_1 \dots b_7$.

算法 2 BytesToBits(**bytes**):字节数组转换为比特数组.

输入: 字节数组 $\mathbf{bytes} \in \mathbb{B}^t$.

输出: 比特数组 $\mathbf{b} \in \{0, 1\}^{8t}$.

```
1: c  $\leftarrow \mathbf{bytes}$ 
2: for ( $i = 0; i < t; i++$ ) do
3:   for ( $j = 0; j < 8; j++$ ) do
4:      $\mathbf{b}[8i + j] \leftarrow \mathbf{c}[i] \bmod 2$ 
5:      $\mathbf{c}[i] \leftarrow \lfloor \mathbf{c}[i] / 2 \rfloor$ 
6:   end for
7: end for
8: return b
```

整数数组与字节数组 设 d 为正整数, 算法3的功能是将整数数组转换为字节数组, 其中整数数组的元素定义在区间 $[0, 2^d - 1]$ 之中, 数组的长度为 n . 在Loong密钥封装算法中, n 是8的倍数. 设整数数组 $\mathbf{c}$ 的某个元素为 $b_0 + b_1 \cdot 2 + \dots + b_{d-1} \cdot 2^{d-1}$, 则它对应的比特串为 $b_0 b_1 \dots b_{d-1}$. 函数ByteEncode首先将长度为 n 的整数数组 $c_0, c_1, \dots, c_{n-1}$ 转换为比特串 $b_0, b_1, \dots, b_{nd-1}$, 然后再利用BitsToBytes函数把比特串转换为字节数组.

算法 3 ByteEncode($\mathbf{c}, n, d$): 整数数组转换为字节数组.

输入: 整数数组 $\mathbf{c} \in \mathbb{Z}_{2^d}^n$, 数组长度 n , 正整数 d .

输出: 字节数组 $\mathbf{bytes} \in \mathbb{B}^{nd/8}$.

```

1:  $\mathbf{b} \leftarrow (0, 0, \dots, 0)$   $\triangleright$   $\mathbf{b}$  的长度为  $nd$ .
2: for ( $i = 0; i < n; i++$ ) do
3:    $a \leftarrow \mathbf{c}_i$ 
4:   for ( $j = 0; j < d; j++$ ) do
5:      $\mathbf{b}[i \cdot d + j] \leftarrow a \bmod 2$ 
6:      $a \leftarrow (a - \mathbf{b}[i \cdot d + j]) / 2$ 
7:   end for
8: end for
    $\mathbf{bytes} \leftarrow \text{BitsToBytes}(\mathbf{b})$ 
9: return  $\mathbf{bytes}$ 

```

算法4的功能是将字节数组转换为整数数组. 首先利用BytesToBits函数将字节数组转换为比特串, 比特串的排序为小端位序. 然后再将比特串转换为整数数组.

算法 4 ByteDecode($\mathbf{bytes}, n, d$): 字节数组转换为整数数组.

输入: 字节数组 $\mathbf{bytes} \in \mathbb{B}^{nd/8}$, 正整数 n 和 d .

输出: 整数数组 $\mathbf{c} \in \mathbb{Z}_{2^d}^n$.

```

1:  $\mathbf{b} \leftarrow \text{BytesToBits}(\mathbf{bytes})$ 
2: for ( $i = 0; i < n; i++$ ) do
3:    $\mathbf{c}[i] \leftarrow \sum_{j=0}^{d-1} \mathbf{b}[i \cdot d + j] \cdot 2^j \bmod 2^d$ 
4: end for
5: return  $\mathbf{c}$ 

```

多项式环和反循环矩阵 设 N 为正整数, 我们定义多项式环 $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$, $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^N + 1)$. $\mathcal{R}$ 和 $\mathcal{R}_q$ 的多项式的系数分别定义在 $\mathbb{Z}$ 和 $\mathbb{Z}_q$ 上. 我们用粗体小写字母来表示 $\mathcal{R}_q$ 的元素, 对任意 $\mathbf{c}(X) = c_0 + c_1X + \dots + c_{N-1}X^{N-1} \in \mathcal{R}_q$, 我们通常将它记为 $\mathbf{c} = (c_0, c_1, \dots, c_{N-1})$. 在不引起歧义的前提下, 粗体小写字母有时表示多项式, 有时表示多项式的系数向量. 定义 $\mathbf{c}(X)$ 对应的矩阵为 $\mathbf{C}$, 其中 $\mathbf{C}$ 的第 i ($i = 0, 1, \dots, N-1$) 行元素为多项

式 $X^i \mathbf{c}(X) \bmod X^N + 1$ 的系数, 具体地,

$$\mathbf{C} = \begin{pmatrix} c_0 & c_1 & c_2 & \cdots & c_{N-2} & c_{N-1} \\ -c_{N-1} & c_0 & c_1 & \cdots & c_{N-3} & c_{N-2} \\ -c_{N-2} & -c_{N-1} & c_0 & \cdots & c_{N-4} & c_{N-3} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ -c_1 & -c_2 & -c_3 & \cdots & -c_{N-1} & c_0 \end{pmatrix}.$$

我们称形如 $\mathbf{C}$ 的矩阵为反循环矩阵. 设多项式矩阵

$$\mathbf{b} = \begin{pmatrix} \mathbf{b}_{00} & \mathbf{b}_{01} & \cdots & \mathbf{b}_{0,k-1} \\ \mathbf{b}_{10} & \mathbf{b}_{11} & \cdots & \mathbf{b}_{1,k-1} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{b}_{k-1,0} & \mathbf{b}_{k-1,1} & \cdots & \mathbf{b}_{k-1,k-1} \end{pmatrix},$$

其中 $\mathbf{b}_{ij} \in R_q (0 \leq i \leq k-1, 0 \leq j \leq k-1)$, 我们定义多项式矩阵 $\mathbf{b}$ 对应的矩阵为

$$\mathbf{B} = \begin{pmatrix} \mathbf{B}_{00} & \mathbf{B}_{01} & \cdots & \mathbf{B}_{0,k-1} \\ \mathbf{B}_{10} & \mathbf{B}_{11} & \cdots & \mathbf{B}_{1,k-1} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{B}_{k-1,0} & \mathbf{B}_{k-1,1} & \cdots & \mathbf{B}_{k-1,k-1} \end{pmatrix},$$

其中 $\mathbf{B}_{ij}$ 是多项式 $\mathbf{b}_{ij}$ 对应的反循环矩阵. 我们称形如 $\mathbf{B}$ 的矩阵为分块反循环矩阵, $\mathbf{B}$ 的每一个子块 $\mathbf{B}_{ij} (0 \leq i \leq k-1, 0 \leq j \leq k-1)$ 都是反循环矩阵.

算法5的功能是将多项式转换为反循环矩阵. 算法的输入为多项式系数向量, 输出为多项式对应的反循环矩阵. 算法6的功能是将多项式矩阵转换为分块反循环矩阵, 算法的输入为多项式矩阵, 输出为分块反循环矩阵.

算法 5 PolyToMat(**c**)

输入: 多项式的系数向量 $\mathbf{c} = (c_0, c_1, \dots, c_{N-1})$.

输出: 对应的反循环矩阵 $\mathbf{C}$.

```
1: for  $i = 0$  to  $N - 1$  do
2:   for  $j = 0$  to  $N - 1$  do
3:     if  $j \geq i$  then
4:        $\mathbf{C}[i][j] \leftarrow c[j - i]$ 
5:     else
6:        $\mathbf{C}[i][j] \leftarrow -c[N + j - i]$ 
7:     end if
8:   end for
9: end for
10: return  $\mathbf{C}$ 
```

算法 6 PolysToBlockMat($\mathbf{A}, k_1, k_2$)

输入: $k_1 \times k_2$ 多项式矩阵 $\mathbf{A}$, 矩阵的每个元素 $\mathbf{A}[i][j]$ 是长度为 N 的多项式系数向量; 正整数 k_1, k_2

输出: 分块反循环矩阵 $\mathbf{B} \in \mathbb{Z}_q^{k_1 N \times k_2 N}$

```
for  $i = 0$  to  $k_1 - 1$  do
  for  $j = 0$  to  $k_2 - 1$  do
     $\mathbf{T} \leftarrow \text{PolyToMat}(\mathbf{A}[i][j])$   $\triangleright T$  是  $N \times N$  反循环矩阵
    for  $t_1 = 0$  to  $N - 1$  do
      for  $t_2 = 0$  to  $N - 1$  do
         $\mathbf{B}[i \cdot N + t_1][j \cdot N + t_2] \leftarrow \mathbf{T}[t_1][t_2]$ 
      end for
    end for
  end for
end for
return  $\mathbf{B}$ 
```

$\mathbb{Z}_q^t$ 的向量采样 算法7是向量空间 $\mathbb{Z}_q^t$ 的向量采样算法. 我们使用16位拒绝采样方法实现该算法. 算法输入长度为 $3t$ 的字节数组, 输出向量 $\mathbf{a} =$

$(a_0, a_1, \dots, a_{t-1}) \in \mathbb{Z}_q^t$. 当 $q = 8191$, $\text{threshold} = 8191 \times 8 = 65528$, 单次拒绝概率为 $1 - 65528/2^{16} = 0.0001220703125$. 当输入为 $3t$ 个字节时, 最大的尝试次数为 $3t/2$, 采样失败的概率为

$$1 - \sum_{i=t}^{3t/2} \binom{3t/2}{i} \left(\frac{8191}{8192}\right)^i \left(\frac{1}{8192}\right)^{3t/2-i}.$$

在Loong密钥封装算法中, t 的值较大, 因此采样失败的概率远小于 2^{-256} .

算法 7 $\text{SampleVec}(\text{bytes}, t)$: $\mathbb{Z}_q^t$ 的向量采样.

输入: 正整数 t 和字符串 $\text{bytes} \in \mathbb{B}^{3t}$.

输出: $(a_0, a_1, \dots, a_{t-1}) \in \mathbb{Z}_q^t$.

```

1:  $j \leftarrow 0$ 
2: for ( $i = 0; i < 3t$  and  $j < t; i = i + 2$ ) do
3:    $x \leftarrow \text{bytes}[i] + 2^8 \cdot \text{bytes}[i + 1]$ 
4:   if ( $x < \text{threshold}$ ) then
5:      $a_j \leftarrow x \bmod q$ 
6:      $j \leftarrow j + 1$ 
7:   end if
8: end for
9: return  $(a_0, a_1, \dots, a_{t-1})$ 

```

普通方阵采样 设

$$\mathbf{A} = \begin{pmatrix} a_{00} & a_{01} & \cdots & a_{0,k-1} \\ a_{10} & a_{11} & \cdots & a_{1,k-1} \\ \vdots & \vdots & \ddots & \vdots \\ a_{k-1,0} & a_{k-1,1} & \cdots & a_{k-1,k-1} \end{pmatrix} \in \mathbb{Z}_q^{k \times k}$$

是没有代数结构的普通矩阵, 我们用一维向量

$$\mathbf{a} = (a_{00}, \dots, a_{0,k-1}, a_{1,0}, \dots, a_{1,k-1}, \dots, a_{k-1,k-1}) \in \mathbb{Z}_q^{k^2}$$

来表示矩阵 $\mathbf{A}$, 则无代数结构的普通矩阵的采样算法如算法8所示. 算法的输入为字节数组 bytes 和正整数 k , 其中数组的长度为 $3k^2$, 输出为矩阵的元素 $\mathbf{a}$.

算法 8 $\text{SampleMat}(\text{bytes}, k)$: 普通矩阵采样.

输入: 正整数 k , 字符串 $\text{bytes} \in \mathbb{B}^{3k^2}$.

输出: $\mathbf{a} = (a_{00}, \dots, a_{0,k-1}, a_{1,0}, \dots, a_{1,k-1}, \dots, a_{k-1,k-1}) \in \mathbb{Z}_q^{k^2}$.

1: $\mathbf{a} \leftarrow \text{SampleVec}(\text{bytes}, k^2)$

2: **return** $\mathbf{a}$

多项式矩阵采样 设多项式矩阵为

$$\mathbf{B} = \begin{pmatrix} \mathbf{b}_{00} & \mathbf{b}_{01} & \cdots & \mathbf{b}_{0,v-1} \\ \mathbf{b}_{10} & \mathbf{b}_{11} & \cdots & \mathbf{b}_{1,v-1} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{b}_{u-1,0} & \mathbf{b}_{u-1,1} & \cdots & \mathbf{b}_{u-1,v-1} \end{pmatrix},$$

其中 $\mathbf{b}_{ij} \in R_q (0 \leq i < u, 0 \leq j \leq v-1)$. 我们将多项式的系数排列成一维向量:

$$\mathbf{b} = (\mathbf{b}_{00}, \dots, \mathbf{b}_{0,v-1}, \mathbf{b}_{1,0}, \dots, \mathbf{b}_{1,v-1}, \dots, \mathbf{b}_{u-1,v-1}) \in \mathbb{Z}_q^{uvN}.$$

则多项式矩阵采样算法如下:

算法 9 $\text{SamplePolyMat}(\text{bytes}, u, v)$: 多项式矩阵采样.

输入: 正整数 u 和 v , 字符串 $\text{bytes} \in \mathbb{B}^{3uvN}$.

输出: $\mathbf{b} = (\mathbf{b}_{00}, \dots, \mathbf{b}_{0,v-1}, \mathbf{b}_{1,0}, \dots, \mathbf{b}_{1,v-1}, \dots, \mathbf{b}_{u-1,v-1}) \in \mathbb{Z}_q^{uvN}$.

1: $\mathbf{b} \leftarrow \text{SampleVec}(\text{bytes}, uvN)$

2: **return** $\mathbf{b}$

噪声向量采样 Loong密钥封装算法的噪声多项式的系数根据中心二项分布来采样. 设 η 为正整数, 算法10是噪声向量采样算法. 算法的输入为 $2t\eta/8$ 个字节长的数组 $\mathbf{b}$, 输出噪声向量 $\mathbf{e} \in \mathbb{Z}_q^t$.

算法 10 `SampleNoiseVec(b,  $t$ )`: 噪声多项式采样.

输入: 正整数 t 和字节数组 $\mathbf{b} \in \mathbb{B}^{2t\eta/8}$.

输出: 噪声向量 $\mathbf{e} \in \mathbb{Z}_q^t$.

```

1:  $(b_0, b_1, \dots, b_{2t\eta-1}) \leftarrow \text{BytesToBits}(\mathbf{B})$ 
2: for  $(i = 0; i < t; i++)$  do
3:    $a \leftarrow \sum_{j=0}^{\eta-1} b_{2i\eta+j}$ 
4:    $b \leftarrow \sum_{j=0}^{\eta-1} b_{2i\eta+\eta+j}$ 
5:    $e_i \leftarrow a - b$ 
6: end for
7: return  $\mathbf{e} = (e_0, e_1, \dots, e_{t-1})$ .
```

噪声矩阵采样 设

$$\mathbf{E} = \begin{pmatrix} e_{00} & e_{01} & \cdots & e_{0,v-1} \\ e_{10} & e_{11} & \cdots & e_{1,v-1} \\ \vdots & \vdots & \ddots & \vdots \\ e_{u-1,0} & e_{u-1,1} & \cdots & e_{u-1,v-1} \end{pmatrix} \in \mathbb{Z}_q^{u \times v}$$

是噪声矩阵, 矩阵的元素服从中心二项分布, 我们可用一维向量

$$\mathbf{e} = (e_{00}, \dots, e_{0,v-1}, e_{1,0}, \dots, e_{1,v-1}, \dots, e_{u-1,v-1}) \in \mathbb{Z}_q^{uv}$$

来表示矩阵 $\mathbf{E}$, 则噪声矩阵的采样算法如算法11所示. 算法的输入为字节数组`bytes`和正整数 u, v , 其中数组的长度为 $2uv\eta/8$. 输出为噪声矩阵的元素组成的向量 $\mathbf{e}$.

算法 11 `SampleNoiseMat(bytes,  $u$ ,  $v$ )`: 普通噪声矩阵采样.

输入: 正整数 u, v , 字符串`bytes` $\in \mathbb{B}^{2uv\eta/8}$.

输出: $\mathbf{e} = (e_{00}, \dots, e_{0,v-1}, e_{1,0}, \dots, e_{1,v-1}, \dots, e_{u-1,v-1}) \in \mathbb{Z}_q^{uv}$.

```

1:  $\mathbf{e} \leftarrow \text{SampleNoiseVec}(\text{bytes}, uv)$ 
2: return  $\mathbf{e}$ 
```

噪声多项式向量采样 设 $\mathbf{e} = (\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_t) \in R^t$ 为噪声多项式向量, 这些向量的系数服从中心二项分布. 噪声多项式向量的采样算法如算法12所示. 算法的输入为字节数组`bytes`, 输出噪声多项式向量的系数所组成的向量 $\mathbf{e}$.

算法 12 `SampleNoisePolyVec(bytes, t)`: 噪声多项式采样.

输入: 正整数 u, v , 字符串 $\mathbf{bytes} \in \mathbb{B}^{2tN\eta/8}$.

输出: $\mathbf{e} = (e_{00}, \dots, e_{0,N-1}, e_{1,0}, \dots, e_{1,N-1}, \dots, e_{t-1,N-1}) \in \mathbb{Z}_q^{tN}$.

1: $\mathbf{e} \leftarrow \text{SampleNoiseVec}(\mathbf{bytes}, tN)$

2: **return** $\mathbf{e}$

多项式向量的编码和解码 设多项式向量 $\mathbf{b} = (\mathbf{b}_0, \mathbf{b}_1, \dots, \mathbf{b}_{k-1}) \in R^k$, 且多项式的系数在区间 $[0, 2^d - 1]$ 之中, 我们将多项式向量的系数看成向量空间 $\mathbb{Z}^{kN}$ 中的向量, 则 $\mathbf{b}$ 的编码算法如算法13所示.

算法 13 `EncodePolyVec(b, k, d)`: 多项式向量编码.

输入: 多项式向量 $\mathbf{b} \in R^k$, 正整数 k 和 d .

输出: 字节数组 $\mathbf{bytes} \in \mathbb{B}^{kNd/8}$.

1: $\mathbf{bytes} \leftarrow \text{ByteEncode}(\mathbf{b}, kN, d)$

2: **return** $\mathbf{bytes}$

相应的解码算法如算法14所示.

算法 14 `DecodePolyVec(bytes, k, d)`: 多项式向量解码.

输入: 字节数组 $\mathbf{bytes} \in \mathbb{B}^{kNd/8}$, 正整数 k 和 d .

输出: $\mathbf{b} \in R^k$.

1: $(\mathbf{b}_0, \mathbf{b}_1, \dots, \mathbf{b}_{k-1}) \leftarrow \text{ByteDecode}(\mathbf{bytes}, kN, d)$

2: **return** $(\mathbf{b}_0, \mathbf{b}_1, \dots, \mathbf{b}_{k-1})$

普通矩阵的编码和解码 设

$$\mathbf{A} = \begin{pmatrix} a_{00} & a_{01} & \cdots & a_{0,\ell-1} \\ a_{10} & a_{11} & \cdots & a_{1,\ell-1} \\ \vdots & \vdots & \ddots & \vdots \\ a_{k-1,0} & a_{k-1,1} & \cdots & a_{k-1,\ell-1} \end{pmatrix} \in \mathbb{Z}_{2^d}^{k \times \ell}$$

是没有代数结构的普通矩阵, 且 $a_{ij} \in [0, 2^d - 1] (0 \leq i < k, 0 \leq j < \ell)$. 我们用一维向量

$$\mathbf{a} = (a_{00}, \dots, a_{0,\ell-1}, a_{1,0}, \dots, a_{1,\ell-1}, \dots, a_{k-1,\ell-1}) \in \mathbb{Z}_{2^d}^{k\ell}$$

来表示矩阵 $\mathbf{A}$, 则无代数结构的普通矩阵的编码如算法15所示.

算法 15 EncodeMat($\mathbf{a}, k, \ell, d$): 多项式向量编码.

输入: 矩阵 $\mathbf{A}$ 的元素 $\mathbf{a} \in \mathbb{Z}_{2^d}^{k\ell}$, 正整数 k, ℓ 和 d .

输出: 字节数组 $\mathbf{bytes} \in \mathbb{B}^{k\ell d/8}$.

```
1:  $\mathbf{bytes} \leftarrow \text{ByteEncode}(\mathbf{b}, k\ell, d)$ 
2: return  $\mathbf{bytes}$ 
```

相应的解码算法如算法16所示.

算法 16 DecodeMat($\mathbf{bytes}, k, \ell, d$): 多项式向量解码.

输入: 字节数组 $\mathbf{bytes} \in \mathbb{B}^{k\ell d/8}$, 正整数 k, ℓ 和 d .

输出: $\mathbf{a} \in \mathbb{Z}_{2^d}^{k\ell}$.

```
1:  $\mathbf{a} \leftarrow \text{ByteDecode}(\mathbf{bytes}, k\ell, d)$ 
2: return  $\mathbf{a}$ 
```

噪声多项式向量的编码和解码 设 $\mathbf{e} = (\mathbf{e}_0, \mathbf{e}_1, \dots, \mathbf{e}_{k-1}) \in R^k$ 为多项式向量, 且 $\|\mathbf{e}_i\|_\infty \leq \eta \leq 6 (0 \leq i < k)$. 设 $d_\eta = 4$. 我们把多项式向量的系数按下标顺序表示成一维数组 $(e_0, e_1, \dots, e_{kN-1}) \in \mathbb{Z}^{kN}$. 噪声多项式向量的编码算法如算法17所示. 算法18为噪声多项式解码算法.

算法 17 EncodeNoisePolyVec($\mathbf{e}, k, d_\eta$)

输入: 正整数 d_η, k ; 噪声多项式向量的系数 $(e_0, e_1, \dots, e_{kN-1})$

输出: 字节数组 $\mathbf{bytes}$.

```
1:  $\mathbf{b} \leftarrow \{0, 0, \dots, 0\}$   $\triangleright$   $\mathbf{b}$ 的长度为 $kN$ .
2: for  $i = 0$  to  $kN - 1$  do
3:    $\mathbf{b}[i] \leftarrow e_i + \eta$ 
4: end for
5:  $\mathbf{bytes} \leftarrow \text{ByteEncode}(\mathbf{b}, kN, d_\eta)$ 
6: return  $\mathbf{bytes}$ 
```

辅助密码函数 Loong密钥封装算法需要用到密码学安全的伪随机函数RNG、杂凑函数H和G、可扩展输出函数XOF.

算法 18 DecodeNoisePolyVec(**bytes**, k, η)

输入: 字节数组**bytes**; 正整数 k, d_η .

输出: 长度为 kN 的数组**e**, 且 $\|\mathbf{e}\|_\infty \leq \eta$.

1: **a** $\leftarrow$ ByteDecode(**bytes**, kN, d_η)

2: **for** $i = 0$ to $kN - 1$ **do**

3: $e_i \leftarrow \mathbf{a}[i] - \eta$

4: **end for**

5: **return** **e** = $(e_0, e_1, \dots, e_{kN-1})$

2.2 Loong公钥加密算法

本小节介绍CPA-安全的Loong公钥加密算法.

2.2.1 公钥和私钥

算法19是Loong公钥和私钥生成算法.

算法 19 Loong_PKE_KeyGen(): Key generation algorithm

输入: NULL.

输出: pk and sk.

```

1:  $seed \in \mathbb{B}^{\lambda/8+16} \leftarrow \text{RNG}()$ 
2:  $(\rho, \sigma, \gamma) \in \mathbb{B}^{\lambda/8+16} \times \mathbb{B}^{\lambda/8+16} \times \mathbb{B}^{\lambda/8+16} \leftarrow \text{XOF}(seed)$ 
3:  $\mathbf{x} \leftarrow \text{XOF}(\rho)$   $\triangleright |\mathbf{x}| = 3(k_1^2 N + 2k_1 k_2 N + k_2^2 N^2).$ 
4:  $\mathbf{A}_1 \leftarrow \text{SamplePolyMat}(\mathbf{x}, k_1, k_1)$   $\triangleright \mathbf{A}_1 \in R_q^{k_1 \times k_1}.$ 
5:  $\mathbf{A}_2 \leftarrow \text{SamplePolyMat}(\mathbf{x} + 3k_1^2 N, k_1, k_2)$   $\triangleright \mathbf{A}_2 \in R_q^{k_1 \times k_2}.$ 
6:  $\mathbf{A}_3 \leftarrow \text{SamplePolyMat}(\mathbf{x} + 3(k_1^2 N + k_1 k_2 N), k_2, k_1)$   $\triangleright \mathbf{A}_3 \in R_q^{k_2 \times k_1}.$ 
7:  $\mathbf{A}_4 \leftarrow \text{SampleMat}(\mathbf{x} + 3(k_1^2 N + 2k_1 k_2 N), k_2 N, k_2 N)$   $\triangleright \mathbf{A}_4 \in \mathbb{Z}_q^{k_2 N \times k_2 N}.$ 
8:  $\mathbf{y} \leftarrow \text{XOF}(\sigma)$   $\triangleright |\mathbf{y}| = 2(k_1 N + k_2 N)\eta/8.$ 
9:  $\mathbf{s}_1 \leftarrow \text{SampleNoisePolyVec}(\mathbf{y}, k_1)$   $\triangleright \mathbf{s}_1 \in R_q^{k_1}.$ 
10:  $\mathbf{s}_2 \leftarrow \text{SampleNoisePolyVec}(\mathbf{y} + 2k_1 N\eta/8, k_2)$   $\triangleright \mathbf{s}_2 \in R_q^{k_2}.$ 
11:  $\mathbf{S}_2 \leftarrow \text{PolysToBlockMat}(\mathbf{s}_2, k_2, 1)$   $\triangleright \mathbf{S}_2 \in \mathbb{Z}_q^{k_2 N \times N}.$ 
12:  $\mathbf{z} \leftarrow \text{XOF}(\gamma)$   $\triangleright |\mathbf{z}| = 2(k_1 N + k_2 N^2)\eta/8.$ 
13:  $\mathbf{e}_1 \leftarrow \text{SampleNoisePolyVec}(\mathbf{z}, k_1)$   $\triangleright \mathbf{e}_1 \in R_q^{k_1}.$ 
14:  $\mathbf{E}_2 \leftarrow \text{SampleNoiseMat}(\mathbf{z} + 2k_1 N\eta/8, k_2 N, N)$   $\triangleright \mathbf{E}_2 \in \mathbb{Z}_q^{k_2 N \times N}.$ 
15:  $\mathbf{b}_1 \leftarrow \mathbf{A}_1 \mathbf{s}_1 + \mathbf{A}_2 \mathbf{s}_2 + \mathbf{e}_1$   $\triangleright \mathbf{b}_1 \in R_q^{k_1}.$ 
16:  $\mathbf{B}_2 \leftarrow \text{PolysToBlockMat}(\mathbf{A}_3 \mathbf{s}_1, k_2, 1) + \mathbf{A}_4 \mathbf{S}_2 + \mathbf{E}_2$   $\triangleright \mathbf{B}_2 \in \mathbb{Z}_q^{k_2 N \times N}.$ 
17:  $\mathbf{b} \leftarrow \text{Compress}(\mathbf{b}_1, 2^{d_q-d_b})$ 
18:  $\mathbf{B} \leftarrow \text{Compress}(\mathbf{B}_2, 2^{d_q-d_b})$ 
19:  $\text{pk} \leftarrow \rho || \text{EncodePolyVec}(\mathbf{b}, k_1, d_b) || \text{EncodeMat}(\mathbf{B}, k_2 N, N, d_b)$ 
20:  $\text{sk} \leftarrow \text{EncodeNoisePolyVec}(\mathbf{s}_1, k_1, d_\eta) || \text{EncodeNoisePolyVec}(\mathbf{s}_2, k_2, d_\eta)$ 
21: return (pk, sk)

```

2.2.2 加密

算法20是Loong加密算法.

算法 20 Loong_PKE_Encrypt(pk, M, γ): Loong encryption

输入: pk, $\mathbf{M} \in \{0,1\}^{N \times N}$ and $\gamma = (\gamma_1, \gamma_2, \gamma_3) \in \mathbb{B}^{\lambda/8+16} \times \mathbb{B}^{\lambda/8+16} \times \mathbb{B}^{\lambda/8+16}$.

输出: Ciphertext $\mathbf{c}$.

```

1:  $\rho \leftarrow \text{pk}[0 : \lambda/8 + 16 - 1]$ 
2:  $\mathbf{x} \leftarrow \text{XOF}(\rho)$   $\triangleright |\mathbf{x}| = 3(k_1^2 N + 2k_1 k_2 N + k_2^2 N^2)$ .
3:  $\mathbf{A}_1 \leftarrow \text{SamplePolyMat}(\mathbf{x}, k_1, k_1)$   $\triangleright \mathbf{A}_1 \in R_q^{k_1 \times k_1}$ .
4:  $\mathbf{A}_2 \leftarrow \text{SamplePolyMat}(\mathbf{x} + 3k_1^2 N, k_1, k_2)$   $\triangleright \mathbf{A}_2 \in R_q^{k_1 \times k_2}$ .
5:  $\mathbf{A}_3 \leftarrow \text{SamplePolyMat}(\mathbf{x} + 3(k_1^2 N + k_1 k_2 N), k_2, k_1)$   $\triangleright \mathbf{A}_3 \in R_q^{k_2 \times k_1}$ .
6:  $\mathbf{A}_4 \leftarrow \text{SampleMat}(\mathbf{x} + 3(k_1^2 N + 2k_1 k_2 N), k_2 N, k_2 N)$   $\triangleright \mathbf{A}_4 \in \mathbb{Z}_q^{k_2 N \times k_2 N}$ .
7:  $\tilde{\mathbf{b}}_1 \leftarrow \text{DecodePolyVec}(\text{pk}[\lambda/8 + 16 : \lambda/8 + 16 + k N d_b / 8 - 1], k_1, d_b)$ 
8:  $\mathbf{b}_1 \leftarrow \text{Decompress}(\tilde{\mathbf{b}}_1, 2^{d_q - d_b})$   $\triangleright \mathbf{b}_1 \in R_q^{k_1}$ .
9:  $\tilde{\mathbf{B}}_2 \leftarrow \text{DecodeMat}(\text{pk}[\lambda/8 + 16 + k N d_b / 8 - 1 : \#pk], k_2 N, k_2 N, d_b)$ 
10:  $\mathbf{B}_2 \leftarrow \text{Decompress}(\tilde{\mathbf{B}}_2, 2^{d_q - d_b})$   $\triangleright \mathbf{B}_2 \in \mathbb{Z}_q^{k_2 N \times N}$ .
11:  $\mathbf{w} \leftarrow \text{XOF}(\gamma_1)$   $\triangleright |\mathbf{w}| = 2(k_1 N + k_2 N)\eta/8$ .
12:  $\mathbf{r}_1 \leftarrow \text{SampleNoisePolyVec}(\mathbf{w}, k_1)$   $\triangleright \mathbf{r}_1 \in R_q^{k_1}$ .
13:  $\mathbf{r}_2 \leftarrow \text{SampleNoisePolyVec}(\mathbf{w} + 2k_1 N \eta / 8, k_2)$   $\triangleright \mathbf{r}_2 \in R_q^{k_2}$ .
14:  $\mathbf{R}_2 \leftarrow \text{PolysToBlockMat}(\mathbf{r}_2^T, 1, k_2)$   $\triangleright \mathbf{R}_2 \in \mathbb{Z}_q^{N \times k_2 N}$ .
15:  $\mathbf{y} \leftarrow \text{XOF}(\gamma_2)$   $\triangleright |\mathbf{y}| = 2(k_1 N + k_2 N^2)\eta/8$ .
16:  $\mathbf{e}_3 \leftarrow \text{SampleNoisePolyVec}(\mathbf{y}, k_1)$   $\triangleright \mathbf{e}_3 \in R_q^{k_1}$ .
17:  $\mathbf{E}_4 \leftarrow \text{SampleNoiseMat}(\mathbf{y} + 2k_1 N \eta / 8, N, k_2 N)$   $\triangleright \mathbf{E}_4 \in \mathbb{Z}_q^{N \times k_2 N}$ .
18:  $\mathbf{u}' \leftarrow \mathbf{r}_1^T \mathbf{A}_1 + \mathbf{r}_2^T \mathbf{A}_3 + \mathbf{e}_3$   $\triangleright \mathbf{u}' \in R_q^{k_1}$ .
19:  $\mathbf{U}' \leftarrow \text{PolysToBlockMat}(\mathbf{r}_1^T \mathbf{A}_2, 1, k_2) + \mathbf{R}_2 \mathbf{A}_4 + \mathbf{E}_4$   $\triangleright \mathbf{U}' \in \mathbb{Z}_q^{N \times k_2 N}$ .
20:  $\mathbf{z} \leftarrow \text{XOF}(\gamma_3)$   $\triangleright |\mathbf{z}| = 2N^2 \eta / 8$ .
21:  $\mathbf{E}_5 \leftarrow \text{SampleNoiseMat}(\mathbf{z}, N, N)$   $\triangleright \mathbf{E}_5 \in \mathbb{Z}_q^{N \times N}$ .
22:  $\mathbf{V}' \leftarrow \text{PolyToMat}(\mathbf{r}_1^T \mathbf{b}_1) + \mathbf{R}_2 \mathbf{B}_2 + \mathbf{E}_5 + \frac{q+1}{2} \cdot \mathbf{M}$   $\triangleright \mathbf{V}' \in \mathbb{Z}_q^{N \times N}$ .
23:  $\mathbf{u} \leftarrow \text{Compress}(\mathbf{u}', 2^{d_q - d_u})$ 
24:  $\mathbf{U} \leftarrow \text{Compress}(\mathbf{U}', 2^{d_q - d_u})$ 
25:  $\mathbf{V} \leftarrow \text{Compress}(\mathbf{V}', 2^{d_q - d_v})$ 
26:  $\mathbf{c} \leftarrow \text{EncodePolyVec}(\mathbf{u}, k_1, d_u) \parallel \text{EncodeMat}(\mathbf{U}, N, k_2 N, d_u) \parallel \text{EncodeMat}(\mathbf{V}, N, N, d_v)$ 
27: return  $\mathbf{c}$ .
```

2.2.3 解密

算法21是Loong解密算法. 解密的过程与NTRU 类似,解密速度很快.

算法 21 Loong_PKE_Decrypt(sk, c): Loong decryption

输入: sk and c.

输出: Plaintext M.

```

1:  $\mathbf{u} \leftarrow \text{DecodePolyVec}(\mathbf{c}, k_1, d_u)$   $\triangleright \mathbf{u} \in R_q^{k_1}$ .
2:  $\mathbf{U} \leftarrow \text{DecodeMat}(\mathbf{c} + k_1 N d_u / 8, N, k_2 N, d_u)$   $\triangleright \mathbf{U} \in \mathbb{Z}_q^{N \times k_2 N}$ .
3:  $\mathbf{V} \leftarrow \text{DecodeMat}(\mathbf{c} + (k_1 N + k_2 N^2) d_u / 8, N, N, d_v)$   $\triangleright \mathbf{V} \in \mathbb{Z}_q^{N \times N}$ .
4:  $(\mathbf{s}_1, \mathbf{s}_2) \leftarrow \text{DecodeNoisePolyVec}(\text{sk}, k_1 + k_2, \eta)$   $\triangleright \mathbf{s}_1 \in R_q^{k_1}, \mathbf{s}_2 \in R_q^{k_2}$ .
5:  $\mathbf{S}_2 \leftarrow \text{PolysToBlockMat}(\mathbf{s}_2, k_2, 1)$   $\triangleright \mathbf{S}_2 \in \mathbb{Z}_q^{k_2 N \times N}$ .
6:  $\mathbf{u}' \leftarrow \text{Decompress}(\mathbf{u}, 2^{d_q - d_u})$ .
7:  $\mathbf{U}' \leftarrow \text{Decompress}(\mathbf{U}, 2^{d_q - d_u})$ .
8:  $\mathbf{V}' \leftarrow \text{Decompress}(\mathbf{V}, 2^{d_q - d_v})$ .
9:  $\mathbf{W} \leftarrow \text{PolyToMat}(\mathbf{u}'^T \mathbf{s}_1) + \mathbf{U}' \mathbf{S}_2$   $\triangleright \mathbf{W} \in \mathbb{Z}_q^{N \times N}$ .
10:  $\mathbf{M} \leftarrow (2 \cdot (\mathbf{V}' - \mathbf{W}) \bmod \pm q) \bmod 2$ 
11: return M

```

正确性 因为 $\mathbf{u}', \mathbf{U}', \mathbf{V}', \mathbf{b}_1, \mathbf{B}_2$ 是经过压缩和解压缩函数作用后所得的值, 根据压缩和解压缩函数的性质, 我们有 $\mathbf{V}' = \text{PolyToMat}(\mathbf{r}_1^T \mathbf{b}_1) + \mathbf{R}_2 \mathbf{B}_2 + \mathbf{E}_5 + \frac{q+1}{2} \cdot \mathbf{M} + \mathbf{E}_6$, $\mathbf{u}' = \mathbf{r}_1^T \mathbf{A}_1 + \mathbf{r}_2^T \mathbf{A}_3 + \mathbf{e}_3 + \mathbf{e}_7$, $\mathbf{U}' = \text{PolysToBlockMat}(\mathbf{r}_1^T \mathbf{A}_2, 1, k_2) + \mathbf{R}_2 \mathbf{A}_4 + \mathbf{E}_4 + \mathbf{E}_8$, $\mathbf{b}_1 = \mathbf{A}_1 \mathbf{s}_1 + \mathbf{A}_2 \mathbf{s}_2 + \mathbf{e}_1 + \mathbf{e}_9$, $\mathbf{B}_2 = \text{PolysToBlockMat}(\mathbf{A}_3 \mathbf{s}_1, k_2, 1) + \mathbf{A}_4 \mathbf{S}_2 + \mathbf{E}_2 + \mathbf{E}_{10}$, 其中 $\|\mathbf{E}_6\|_\infty \leq 2^{d_q - d_v - 1}$, $\|\mathbf{e}_7\|_\infty \leq 2^{d_q - d_u - 1}$, $\|\mathbf{E}_8\|_\infty \leq 2^{d_q - d_u - 1}$, $\|\mathbf{e}_9\|_\infty \leq 2^{d_q - d_b - 1}$, $\|\mathbf{E}_{10}\|_\infty \leq 2^{d_q - d_b - 1}$. 设多项式向量 $\mathbf{s}_1, \mathbf{r}_1^T, \mathbf{r}_2^T, \mathbf{e}_3, \mathbf{e}_7, \mathbf{b}_1$ 对应的矩阵为 $\mathbf{S}_1, \mathbf{R}_1, \mathbf{R}_2, \mathbf{E}_3, \mathbf{E}_7, \mathbf{B}_1$. 于是我们有

$$\begin{aligned}
& 2 \cdot (\mathbf{V}' - \mathbf{W}) \bmod \pm q \\
= & 2 \cdot (\mathbf{R}_1 \mathbf{B}_1 + \mathbf{R}_2 \mathbf{B}_2 + \mathbf{E}_5 + \Delta \mathbf{M} + \mathbf{E}_6 - \mathbf{R}_1 \mathbf{A}_1 \mathbf{S}_1 - \mathbf{R}_2 \mathbf{A}_3 \mathbf{S}_1 - (\mathbf{E}_3 + \mathbf{E}_7) \mathbf{S}_1) \\
& - 2 \cdot (\mathbf{R}_1 \mathbf{A}_2 \mathbf{S}_2 - \mathbf{R}_2 \mathbf{A}_4 \mathbf{S}_2 - (\mathbf{E}_4 + \mathbf{E}_8) \mathbf{S}_2) \bmod \pm q \\
= & 2 \cdot (\mathbf{R}_1 \mathbf{A}_1 \mathbf{S}_1 + \mathbf{R}_1 \mathbf{A}_2 \mathbf{S}_2 + \mathbf{R}_1 (\mathbf{E}_1 + \mathbf{E}_9) + \mathbf{R}_2 \mathbf{A}_3 \mathbf{S}_1 + \mathbf{R}_2 \mathbf{A}_4 \mathbf{S}_2 + \mathbf{R}_2 (\mathbf{E}_2 + \mathbf{E}_{10})) \\
& + 2 \cdot (\mathbf{E}_5 + \Delta \mathbf{M} + \mathbf{E}_6 - \mathbf{R}_1 \mathbf{A}_1 \mathbf{S}_1 - \mathbf{R}_2 \mathbf{A}_3 \mathbf{S}_1 - (\mathbf{E}_3 + \mathbf{E}_7) \mathbf{S}_1 - \mathbf{R}_1 \mathbf{A}_2 \mathbf{S}_2 - \mathbf{R}_2 \mathbf{A}_4 \mathbf{S}_2) \\
& - 2 \cdot ((\mathbf{E}_4 + \mathbf{E}_8) \mathbf{S}_2) \bmod \pm q \\
= & 2 \cdot (\mathbf{R}_1 (\mathbf{E}_1 + \mathbf{E}_9) + \mathbf{R}_2 (\mathbf{E}_2 + \mathbf{E}_{10}) + \mathbf{E}_5 + \mathbf{E}_6 - (\mathbf{E}_3 + \mathbf{E}_7) \mathbf{S}_1 - (\mathbf{E}_4 + \mathbf{E}_8) \mathbf{S}_2) + \mathbf{M} \bmod \pm q
\end{aligned}$$

如果

$$\|2 \cdot (\mathbf{R}_1 (\mathbf{E}_1 + \mathbf{E}_9) + \mathbf{R}_2 (\mathbf{E}_2 + \mathbf{E}_{10}) + \mathbf{E}_5 + \mathbf{E}_6 - (\mathbf{E}_3 + \mathbf{E}_7) \mathbf{S}_1 - (\mathbf{E}_4 + \mathbf{E}_8) \mathbf{S}_2) + \mathbf{M}\|_\infty < \frac{q}{2},$$

那么

$$\begin{aligned} & 2 \cdot (\mathbf{R}_1(\mathbf{E}_1 + \mathbf{E}_9) + \mathbf{R}_2(\mathbf{E}_2 + \mathbf{E}_{10}) + \mathbf{E}_5 + \mathbf{E}_6 - (\mathbf{E}_3 + \mathbf{E}_7)\mathbf{S}_1 - (\mathbf{E}_4 + \mathbf{E}_8)\mathbf{S}_2) + \mathbf{M} \bmod^{\pm} q \\ = & 2 \cdot (\mathbf{R}_1(\mathbf{E}_1 + \mathbf{E}_9) + \mathbf{R}_2(\mathbf{E}_2 + \mathbf{E}_{10}) + \mathbf{E}_5 + \mathbf{E}_6 - (\mathbf{E}_3 + \mathbf{E}_7)\mathbf{S}_1 - (\mathbf{E}_4 + \mathbf{E}_8)\mathbf{S}_2) + \mathbf{M} \end{aligned}$$

于是, 我们就可以成功地解密密文.

解密失败的概率 当

$$\|\mathbf{R}_1(\mathbf{E}_1 + \mathbf{E}_9) + \mathbf{R}_2(\mathbf{E}_2 + \mathbf{E}_{10}) + \mathbf{E}_5 + \mathbf{E}_6 - (\mathbf{E}_3 + \mathbf{E}_7)\mathbf{S}_1 - (\mathbf{E}_4 + \mathbf{E}_8)\mathbf{S}_2\|_{\infty} \geq \frac{q}{4}$$

时, 解密失败. 我们接下来计算解密失败的概率. 记

$$\mathbf{E} = \mathbf{R}_1(\mathbf{E}_1 + \mathbf{E}_9) + \mathbf{R}_2(\mathbf{E}_2 + \mathbf{E}_{10}) + \mathbf{E}_5 + \mathbf{E}_6 - (\mathbf{E}_3 + \mathbf{E}_7)\mathbf{S}_1 - (\mathbf{E}_4 + \mathbf{E}_8)\mathbf{S}_2,$$

我们可将矩阵 $\mathbf{E}$ 改写成

$$\mathbf{E} = \begin{pmatrix} \mathbf{R}_1 & \mathbf{R}_2 \end{pmatrix} \begin{pmatrix} \mathbf{E}_1 \\ \mathbf{E}_2 \end{pmatrix} + \begin{pmatrix} \mathbf{R}_1 & \mathbf{R}_2 \end{pmatrix} \begin{pmatrix} \mathbf{E}_9 \\ \mathbf{E}_{10} \end{pmatrix} - \begin{pmatrix} \mathbf{E}_3 & \mathbf{E}_4 \end{pmatrix} \begin{pmatrix} \mathbf{S}_1 \\ \mathbf{S}_2 \end{pmatrix} - \begin{pmatrix} \mathbf{E}_7 & \mathbf{E}_8 \end{pmatrix} \begin{pmatrix} \mathbf{S}_1 \\ \mathbf{S}_2 \end{pmatrix} + \mathbf{E}_5 + \mathbf{E}_6.$$

显然, 矩阵 $\mathbf{E}$ 的每个元素都可以表示成以下形式

$$\langle \hat{\mathbf{r}}, \hat{\mathbf{e}}_1 \rangle + \langle \hat{\mathbf{r}}, \hat{\mathbf{e}}_2 \rangle - \langle \hat{\mathbf{e}}_3, \hat{\mathbf{s}} \rangle - \langle \hat{\mathbf{e}}_4, \hat{\mathbf{s}} \rangle + e_5 + e_6,$$

其中 $\hat{\mathbf{r}}, \hat{\mathbf{s}}, \hat{\mathbf{e}}_1, \hat{\mathbf{e}}_3$ 是 $n = (k_1 + k_2)N$ 维向量, 且向量的元素服从中心二项分布 ψ_{η} ; $\hat{\mathbf{e}}_2, \hat{\mathbf{e}}_4$ 是 $n = (k_1 + k_2)N$ 维向量, 我们选择适当的 d_u, d_b 的值, 使得 $\hat{\mathbf{e}}_2, \hat{\mathbf{e}}_4$ 的元素在集合 $\{-\eta, \dots, -1, 0, 1, \dots, \eta\}$ 之中; e_5 的值服从中心二项分布 ψ_{η} ; $\|e_6\|_{\infty} \leq 2^{d_q - d_v - 1}$. 当矩阵 $\mathbf{E}$ 的某个元素

$$\langle \hat{\mathbf{r}}, \hat{\mathbf{e}}_1 \rangle + \langle \hat{\mathbf{r}}, \hat{\mathbf{e}}_2 \rangle - \langle \hat{\mathbf{e}}_3, \hat{\mathbf{s}} \rangle - \langle \hat{\mathbf{e}}_4, \hat{\mathbf{s}} \rangle + e_5 + e_6 \geq \frac{q}{4}$$

时, 解密失败. 或者当

$$\langle \hat{\mathbf{r}}, \hat{\mathbf{e}}_1 \rangle + \langle \hat{\mathbf{r}}, \hat{\mathbf{e}}_2 \rangle - \langle \hat{\mathbf{e}}_3, \hat{\mathbf{s}} \rangle - \langle \hat{\mathbf{e}}_4, \hat{\mathbf{s}} \rangle \geq \frac{q}{4} - 2^{d_q - d_v - 1} - \eta$$

时解密失败. 假设向量 $\hat{\mathbf{r}}, \hat{\mathbf{s}}, \hat{\mathbf{e}}_1, \hat{\mathbf{e}}_2, \hat{\mathbf{e}}_3, \hat{\mathbf{e}}_4$ 均值为0标准差为 σ 的高斯分布, 则

$$\langle \hat{\mathbf{r}}, \hat{\mathbf{e}}_1 \rangle + \langle \hat{\mathbf{r}}, \hat{\mathbf{e}}_2 \rangle - \langle \hat{\mathbf{e}}_3, \hat{\mathbf{s}} \rangle - \langle \hat{\mathbf{e}}_4, \hat{\mathbf{s}} \rangle$$

的值近似服从均值为0标准差为 $\sqrt{4n}\sigma^2$ 的高斯分布, 于是

$$\langle \hat{\mathbf{r}}, \hat{\mathbf{e}}_1 \rangle + \langle \hat{\mathbf{r}}, \hat{\mathbf{e}}_2 \rangle - \langle \hat{\mathbf{e}}_3, \hat{\mathbf{s}} \rangle - \langle \hat{\mathbf{e}}_4, \hat{\mathbf{s}} \rangle \geq \frac{q}{4} - 2^{d_q - d_v - 1} - \eta$$

的概率大约为

$$1 - \text{erf} \left(\frac{\frac{q}{4} - 2^{d_q - d_v - 1} - \eta}{\sqrt{2}(\sqrt{4n}\sigma^2)} \right),$$

其中 $\text{erf}(z) = \frac{2}{\sqrt{\pi}} \int_0^z e^{-t^2} dt$, $\sigma^2 = \eta/2$. 因此, 解密失败的概率大约为

$$n \cdot \left(1 - \text{erf} \left(\frac{\frac{q}{4} - 2^{d_q - d_v - 1} - \eta}{\sqrt{2}(\sqrt{4n}\sigma^2)} \right) \right).$$

2.3 Loong密钥封装算法

我们用Fujisaki - Okamoto变换[12], 将Loong加密算法变成密钥封装算法. 算法22是密钥生成算法, 其中 $\text{pk}_{size} = (\lambda + k_1 N d_b + k_2 N^2 d_b)/8 + 16$, $\text{sk}_{size} = (k_1 + k_2)N/2 + \text{pk}_{size} + |\text{H}(\text{pk})| + \lambda/8 + 16$. 算法23的功能是封装密钥, 其中 ctx_{size} 表示密文的字节长度, $\text{ctx}_{size} = (k_1 N d_u + k_2 N^2 d_u + N^2 d_v)/8$. 算法24的功能是解封密钥.

算法 22 Loong_KEM_KeyGen()

输入: NULL

输出: $\text{pk} \in \mathbb{B}^{\text{pk}_{size}}$ 和 $\text{sk} \in \mathbb{B}^{\text{sk}_{size}}$

1: $\text{salt} \in \mathbb{B}^{\lambda/8+16} \leftarrow \text{RNG}()$

2: $(\text{pk}, \text{sk}') \leftarrow \text{Loong_PKE_KeyGen}()$

▷ 执行算法19.

3: $\text{sk} \leftarrow (\text{sk}' \parallel \text{pk} \parallel \text{H}(\text{pk}) \parallel \text{salt})$

4: **return** (pk, sk)

算法 23 Loong_KEM_Encaps(pk)

输入: 封装密钥 $\text{pk} \in \mathbb{B}^{\text{pk}_{size}}$.

输出: 共享密钥 $K \in \mathbb{B}^{\lambda/8}$ 和密文 $\text{c} \in \mathbb{B}^{\text{ctx}_{size}}$.

1: $\text{M} \in \{0, 1\}^{N \times N} \leftarrow \text{RNG}()$

2: $(K, \gamma) \leftarrow \text{G}(\text{M} \parallel \text{H}(\text{pk}))$

3: $\text{c} \leftarrow \text{Loong_PKE_Encrypt}(\text{pk}, \text{M}, \gamma)$

4: **return** (K, c)

算法 24 Loong_KEM_Decaps($\mathbf{sk}, \mathbf{c}$)

输入: 解封密钥 $\mathbf{sk} \in \mathbb{B}^{\mathbf{sk}_{size}}$ 和密文 $\mathbf{c} \in \mathbb{B}^{\mathbf{ctx}_{size}}$

输出: 共享密钥 $K \in \mathbb{B}^{\lambda/8}$

- 1: $(\mathbf{sk}', \mathbf{pk}, \mathbf{h}, \mathbf{salt}) \leftarrow \mathbf{sk}$
 - 2: $\mathbf{M}' \leftarrow \text{Loong_PKE_Decrypt}(\mathbf{sk}', \mathbf{c})$
 - 3: $(K', \gamma') \leftarrow \mathbf{G}(\mathbf{M}' \parallel \mathbf{h})$
 - 4: $\bar{K} \leftarrow \mathbf{H}(\mathbf{salt} \parallel \mathbf{c})$
 - 5: $\mathbf{c}' \leftarrow \text{Loong_PKE_Encrypt}(\mathbf{pk}, \mathbf{M}', \gamma')$
 - 6: **if** $\mathbf{c} \neq \mathbf{c}'$ **then**
 - 7: $K' \leftarrow \bar{K}$
 - 8: **end if**
 - 9: **return** K'
-

2.4 参数

通过综合考虑安全、性能和解密失败的概率后, 我们选择Loong密钥封装算法的参数如表2所示.

	λ	q	N	(k_1, k_2)	η	(d_b, d_u, d_v)	δ
Loong128	128	8191	12	(48, 4)	5	(10, 10, 4)	2^{-141}
Loong256	256	8191	16	(64, 6)	4	(10, 10, 10)	2^{-161}
Loong384	384	8191	20	(72, 8)	3	(11, 11, 6)	2^{-190}
Loong512	512	8191	24	(80, 10)	2	(11, 11, 4)	2^{-260}

表 2: Loong密钥封装算法的参数.

参数 λ 表示密钥封装算法的经典比特安全强度. Loong128、Loong256、Loong384和Loong512密钥封装算法对应的封装密钥长度和安全强度分别为: 128比特、256比特、384比特和512比特. 参数 $q = 8191 = 2^{13} - 1$ 是一个梅森素数, 可以快速计算模 q 运算. 参数 N 是模多项式的次数, 多项式矩阵和多项式向量的元素为次数小于 N 的多项式. k_1 和 k_2 是多项式矩阵和多项式向量的维数相关的参数. η 是中心二项分布的参数. d_b 是公钥压缩参数, d_u 和 d_v 是密文压缩参数. δ 是密钥解封装失败的概率.

参数选择的说明 LoongKEM的安全性基于半结构化的LWE问题. 我们选择 $N = 12, 16, 20, 24$ 的原因如下: 一方面, 参数 N 的值不能太大, 否则LoongKEM具有和Kyber一样的强代数结构. 另一方面, 参数 N 的值不能太小, 我们希望一个 $N \times N$ 矩阵能够负载至少 λ 比特信息, 这样我们只需用一个矩阵就能负载密钥的信息, 从而提高密钥封装算法的性能和效率, 降低存储空间的大小和传输带宽. 我们选择 $q = 8191 = 2^{13} - 1$ 的原因如下: 8191 是一个梅森素数, 可以快速计算模 q 运算; 此外参数 q 的值必须使得解密失败的概率很低, 根据解密失败的概率公式, 我们选择 $q = 8191$. 我们参数 k_1, k_2 的值, 主要基于以下原因: 首先公钥矩阵的维数为 $(k_1 + k_2)N$, 当 q 和 N 的值选定后, 我们必须选择适当的 k_1, k_2 的值, 使得LoongKEM算法满足安全要求, 我们用5小节中的安全评估公式, 计算满足安全要求的 $(k_1 + k_2)N$ 的值(留有做够的安全冗余, 保守的估计). 此外 k_2 的值决定了公钥矩阵和密文的非结构化程度, 我们希望 $(2\eta + 1)^{k_2} > 2^\lambda$, 使得敌手很难通过穷举的方式移除公钥和密文的非结构化部分. η 的值的选择主要根据5小节中的安全评估公式和解密失败的概率公式, 保证LoongKEM既满足安全需求, 又有较低的解密失败的概率. (d_b, d_u, d_v) 的值主要由解密失败的概率公式决定.

3 设计原理

LoongKEM密钥封装算法的设计原理与Kyber和FrodoKEM有相似之处. LoongKEM与Kyber和FrodoKEM的主要不同如下:

- 解密的逻辑不同. Kyber和FrodoKEM解密算法的最后一步通常是比较密文与 $q/4$ 的大小, 如果大于 $q/4$, 则对应的明文比特为1, 否则对应的明文比特为0. 在Loong加密算法中, 我们只需要做模运算, 就可以恢复明文. 这与NTRU 加密算法的解密过程相似, 提升了解密的速度.
- 压缩和解压缩公钥和密文的方法不同, LoongKEM只需利用移位运算和加法运算就可以实现 $\mathbb{Z}_q$ 的元素的压缩和解压缩, 而不需要除法运算和模运算.
- 基于的基本问题不同. Kyber基于MLWE问题的困难性, FrodoKEM基于LWE问题的困难性, 而LoongKEM基于半结构化LWE问题(SLWE问题)的困难性.

3.1 半结构化的LWE问题

Loong密钥封装算法基于半结构化的LWE问题(简记为SLWE 问题). 我们在此给出SLWE问题的定义. 设 N, q, u, v 为正整数. 设 $\phi(X) \in \mathbb{Z}[X]$, 且 $\phi(X)$ 的次数为 N . 设 $\hat{R} = \mathbb{Z}[X]/\phi(X)$ 为多项式环. 对任意 $\mathbf{f}(X) = f_0 + f_1X + \dots + f_{N-1}X^{N-1} \in \hat{R}$, 定义 $\mathbf{f}(X)$ 对应的矩阵为 $\mathbf{M}_{\mathbf{f}} \in \mathbb{Z}^{N \times N}$, 其中 $\mathbf{M}_{\mathbf{f}}$ 的第 $i(0 \leq i < N)$ 行为多项式 $X^i \mathbf{f}(X) \bmod \phi(X)$ 的系数向量. 设多项式矩阵

$$\mathbf{F} = \begin{pmatrix} \mathbf{f}_{00} & \mathbf{f}_{01} & \cdots & \mathbf{f}_{0,v-1} \\ \mathbf{f}_{10} & \mathbf{f}_{11} & \cdots & \mathbf{f}_{1,v-1} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{f}_{u-1,0} & \mathbf{f}_{u-1,1} & \cdots & \mathbf{f}_{u-1,v-1} \end{pmatrix} \in \hat{R}^{u \times v},$$

其中 $\mathbf{f}_{ij} \in \hat{R}(0 \leq i \leq u-1, 0 \leq j \leq v-1)$, 我们定义多项式矩阵 $\mathbf{F}$ 对应的整数矩阵为

$$\mathbf{M}_{\mathbf{F}} = \begin{pmatrix} \mathbf{M}_{\mathbf{f}_{00}} & \mathbf{M}_{\mathbf{f}_{01}} & \cdots & \mathbf{M}_{\mathbf{f}_{0,v-1}} \\ \mathbf{M}_{\mathbf{f}_{10}} & \mathbf{M}_{\mathbf{f}_{11}} & \cdots & \mathbf{M}_{\mathbf{f}_{1,v-1}} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{M}_{\mathbf{f}_{u-1,0}} & \mathbf{M}_{\mathbf{f}_{u-1,1}} & \cdots & \mathbf{M}_{\mathbf{f}_{u-1,v-1}} \end{pmatrix} \in \mathbb{Z}^{uN \times vN},$$

其中 $\mathbf{M}_{\mathbf{f}_{ij}}$ 是多项式 $\mathbf{f}_{ij}$ 对应的矩阵.

设 k_1, k_2, k_3 为正整数, 矩阵 $\mathbf{A} = \begin{pmatrix} \mathbf{M}_{\mathbf{A}_1} & \mathbf{M}_{\mathbf{A}_2} \\ \mathbf{M}_{\mathbf{A}_3} & \mathbf{A}_4 \end{pmatrix}$, 其中 $\mathbf{M}_{\mathbf{A}_1}, \mathbf{M}_{\mathbf{A}_2}, \mathbf{M}_{\mathbf{A}_3}$ 分别是多项式矩阵 $\mathbf{A}_1 \in \hat{R}^{k_1 \times k_1}, \mathbf{A}_2 \in \hat{R}^{k_1 \times k_2}, \mathbf{A}_3 \in \hat{R}^{k_3 \times k_1}$ 对应的整数矩阵. $\mathbf{A}_4 \in \mathbb{Z}^{k_3N \times k_2N}$ 是没有代数结构的普通整数矩阵. 矩阵 $\mathbf{A}$ 的元素服从均匀分布.

设 χ 为定义在 $\mathbb{Z}$ 上的概率分布, 设 $\mathbf{S} = \begin{pmatrix} \mathbf{M}_{\mathbf{S}_1} \\ \mathbf{M}_{\mathbf{S}_2} \end{pmatrix}$, $\mathbf{E} = \begin{pmatrix} \mathbf{M}_{\mathbf{E}_1} \\ \mathbf{E}_2 \end{pmatrix}$, 其中 $\mathbf{M}_{\mathbf{S}_1}, \mathbf{M}_{\mathbf{S}_2}, \mathbf{M}_{\mathbf{E}_1}$ 分别是多项式矩阵 $\mathbf{S}_1 \in \hat{R}^{k_1 \times 1}, \mathbf{S}_2 \in \hat{R}^{k_2 \times 1}, \mathbf{E}_1 \in \hat{R}^{k_1 \times 1}$ 对应的整数矩阵. $\mathbf{E}_2$ 是 k_3N 行 N 列的没有代数结构的矩阵. 矩阵 $\mathbf{S}$ 和 $\mathbf{E}$ 的元素服从 χ 分布.

定义1 (SLWE问题). 设 $\mathbf{B} = \mathbf{AS} + \mathbf{E} \bmod q$, SLWE问题定义如下: 给定矩阵 $\mathbf{A}$ 和 $\mathbf{B}$, 求矩阵 $\mathbf{S}$.

显然, 从复杂度理论的角度: $LWE \geq SLWE \geq MLWE \geq RLWE$. SLWE问题是一种特殊的LWE问题, MLWE问题是一种特殊的SLWE问题,

RLWE问题是一种特殊的MLWE 问题.

与基于LWE问题的密码方案相比, SLWE问题部分引入了代数结构, 减少了公钥和密文的长度. 与MLWE问题和RLWE问题相比, SLWE问题引入的代数结构相对弱一些. SMLWE问题同时兼顾安全和效率, 因此我们使用SLWE问题来构造密码算法.

3.2 噪声分布

在基于LWE问题的密码系统中, 噪声分布通常有以下几种:

- 离散高斯分布, 例如FrodoKEM[13]等.
- 中心二项分布, 例如Kyber[10] 和Dilithium[11]等.
- NTRU类型分布, 即多项式的系数属于 $\{-1, 0, 1\}$, 且固定 -1 、 0 和 1 的数量. 例如NTRU加密算法[2]等.
- 均匀分布, 例如Saber[14]等.

由于中心二项分布能够在常数时间内快速实现, 可以更好地抵御侧信道攻击, 因此我们用中心二项分布来采样噪声多项式和噪声矩阵.

4 性能分析

4.1 性能

在存储公钥和私钥时, 我们的参考实现只存储生成公钥矩阵 $\mathbf{A}$ 的种子比特, 其长度为 $\lambda/8 + 16$ 个字节. 在带宽和存储空间不受限制的情况下, 我们可以存储整个公钥矩阵 $\mathbf{A}$ 的元素, 以加快密钥封装和解封装的速度.

Loong密钥封装算法的参考实现和AVX2实现的性能如表3所示.

Loong128		
size (Bytes)	ref (10 ³ Cycles)	avx2 (10 ³ Cycles)
sk : 1832	gen : 2621	gen : 2602
pk : 1472	enc : 2647	enc : 2617
ct : 1512	dec : 2676	dec : 2636
Loong256		
size (Bytes)	ref (10 ³ Cycles)	avx2 (10 ³ Cycles)
sk : 3888	gen : 6483	gen : 6426
pk : 3248	enc : 6552	enc : 6477
ct : 3520	dec : 6645	dec : 6531
Loong384		
size (Bytes)	ref (10 ³ Cycles)	avx2 (10 ³ Cycles)
sk : 7365	gen : 20555	gen : 20531
pk : 6444	enc : 20656	enc : 20543
ct : 6680	dec : 20843	dec : 20795
Loong512		
size (Bytes)	ref (10 ³ Cycles)	avx2 (10 ³ Cycles)
sk : 11864	gen : 33405	gen : 33331
pk : 10640	enc : 33756	enc : 35058
ct : 10848	dec : 34006	dec : 33736

表 3: Loong密钥封装算法的性能

参考实现和AVX2实现的测试环境: CPU型号为Intel Core i7-1065G7, CPU的时钟频率为1.30GHz; 性能测试机器的内存大小为8G; 操作系统为Ubuntu 24.04.3 LTS, Linux内核版本为6.14.0; 编译器版本为gcc 13.3.0; 编译选项为-03 -march=native -fomit-frame-pointer; 表格中的Cycles数量为执行相关函数1000次后得到的平均值. 我们的参考实现和AVX2实现都没有使用堆内存.

4.2 测试值

利用商用密码标准研究院提供的辅助程序和数据, 我们生成了所有算法实例的测试向量, 这些测试向量放在提交文件包的Test_Vector目录里, 具体

的测试向量文件为:

```
/ Test_Vector/KAT_KEM_Loong128.txt
/ Test_Vector/KAT_KEM_Loong256.txt
/ Test_Vector/KAT_KEM_Loong384.txt
/ Test_Vector/KAT_KEM_Loong512.txt
```

5 安全分析

5.1 安全证明

设 XOF , H 和 G 是随机预言机, 判定性SLWE问题是困难的, 公钥矩阵 $\mathbf{A}$, $\mathbf{B}$ 没有被压缩, 我们首先证明LoongPKE是IND-CPA安全的.

定理1. 假设判定性SLWE问题是困难的, 则LoongPKE公钥加密算法在选择明文攻击下是IND-CPA安全的.

证明. 假设存在一个概率多项式时间(PPT)敌手 $\mathcal{A}$, 能以不可忽略的优势 ϵ 赢得IND-CPA游戏. 我们构造一个模拟器 $\mathcal{B}$ 来解决判定性SLWE问题. 假设 $\mathcal{B}$ 收到一个判定性SLWE挑战样本 $\mathbf{A}, \mathbf{B}$, 其中 $\mathbf{B}$ 要么是SLWE分布, 即 $\mathbf{B} = \mathbf{A}\mathbf{S} + \mathbf{E}$, 要么是均匀随机矩阵. $\mathcal{B}$ 将挑战中的 $\mathbf{A}, \mathbf{B}$ 直接作为Loong-PKE的公钥发送给 $\mathcal{A}$. 当 $\mathcal{A}$ 提交两个明文 $\mathbf{m}_0, \mathbf{m}_1$ 时, $\mathcal{B}$ 随机选择比特 $\beta \in \{0, 1\}$. $\mathcal{B}$ 再次利用SLWE挑战的数学结构来生成密文 $(\mathbf{U}, \mathbf{V})$. 若 $\mathbf{B}$ 是真实SLWE样本, 则密文 $(\mathbf{U}, \mathbf{V})$ 的分布与真实Loong-PKE加密 $\mathbf{m}_\beta$ 的分布完全一致. 若 $\mathbf{B}$ 是均匀随机的, 则密文在统计上独立于明文 $\mathbf{m}_\beta$. 因此, $\mathcal{A}$ 猜对 β 的优势直接转化为 $\mathcal{B}$ 区分判定性SLWE样本的优势, 即

$$Adv_{IND-CPA}^{Loong-PKE}(\mathcal{A}) \leq 2 \cdot Adv_{SLWE}(\mathcal{B}).$$

□

利用Fujisaki-Okamoto变换, 我们把IND-CPA安全的LoongPKE加密算法变成LoongKEM密钥封装算法. 接下来我们证明LoongKEM在经典随机预言机模型下是紧的IND-CCA安全的, 在量子随机预言机模型下是非紧的IND-CCA安全的. 根据文献[15], 我们有

定理2 (Theorem 3.1 and 3.2 in [15]). 对任意敌手 $\mathcal{B}$, 假设 $\mathcal{B}$ 至多只能发起 q_D 次解密失败查询, q_G 次随机预言机 G 查询, q_H 次随机预言机 H 查询, 存在

敌手 $\mathcal{A}$, 使得

$$Adv_{LoongKEM}^{IND-CCA}(\mathcal{B}) \leq q_H \cdot \delta + (q_H + 2q_G + 1)/|\mathcal{M}| + 3 \cdot Adv_{LoongPKE}^{IND-CPA}(\mathcal{A}),$$

其中 $\mathcal{A}$ 与 $\mathcal{B}$ 所需时间相同.

定理3 (Theorem 4.4 and 4.6 in [15]). 对任意量子敌手 $\mathcal{B}$, 假设 $\mathcal{B}$ 至多只能发起 q_D 次经典的解密失败查询, q_G 次量子随机预言机 $\mathcal{G}$ 查询, q_H 次量子随机预言机 $\mathcal{H}$ 查询, $q_{\hat{H}}$ 次量子随机预言机 $\hat{\mathcal{H}}$ 查询, 则存在量子敌手 $\mathcal{A}$, 使得

$$Adv_{LoongKEM}^{IND-CCA}(\mathcal{B}) \leq (2q_{\hat{H}} + q_H) \cdot \sqrt{8 \cdot \delta(q_G + 1)^2 + (1 + 2q_G) \sqrt{Adv_{LoongPKE}^{IND-CPA}(\mathcal{A})}},$$

其中 $\mathcal{A}$ 与 $\mathcal{B}$ 所需时间相同.

5.2 BKZ算法的复杂度

在对Loong密钥封装算法的安全性做评估之前, 我们首先介绍BKZ格约化算法的计算复杂度. BKZ算法是目前最好的寻找格的短向量的算法. 而在BKZ算法中, 又会用到枚举算法或者筛算法作为子算法. 在实际应用中, 用筛算法作为子算法的效果最好. 因此, 在本文中, 我们假设BKZ算法使用的子算法为筛算法. 设应用筛算法的子格的维数为 β , 根据文献[16, 17, 18, 19], 在经典计算机中, 筛算法的时间复杂度为 $2^{0.292\beta + o(\beta)}$, 空间复杂度为 $2^{0.2075\beta + o(\beta)}$. 根据文献[20, 21, 22, 23], 在量子计算机中, 筛算法的时间复杂度为 $2^{0.257\beta + o(\beta)}$, 同时需要相同数量级的量子空间复杂度. 由于BKZ算法的主要消耗在筛算法中, 因此, 我们将筛算法的计算复杂度作为BKZ算法的计算复杂度.

设格 $\mathcal{L}$ 的维数为 d , 接下来我们确定 β 的大小. 根据文献[8, 24], BKZ算法成功输出目标短向量 $\mathbf{s}$ 的条件为

$$\sqrt{\beta/d} \cdot \|\mathbf{s}\| < \delta_\beta^{2\beta-d-1} \cdot \mathbf{Vol}(\mathcal{L})^{1/d}, \quad (1)$$

其中 δ_β 为BKZ算法的root-Hermite-Factor, 根据文献[25], 当 $\beta > 50$ 时,

$$\delta_\beta = \left(\frac{(\pi\beta)^{1/\beta} \beta}{2\pi e} \right)^{1/(2\beta-2)}.$$

从而我们可以求出满足不等式(1)的最小的 β ($\beta > 50$), 进而评估BKZ算法的计算复杂度.

5.3 Loong密钥封装算法的实际安全评估

Loong密钥封装算法仍然具有一些代数结构, 目前我们不知道如何有效地利用这些代数结构来攻击密码系统, 因此我们暂时忽略这些代数结构. 由于SLWE问题可以看成为原始的LWE问题的一个变种, 因此我们可以用求解LWE问题的算法来求解SLWE问题.

设Loong密钥封装算法的参数为 (q, k_1, k_2, N) , 公钥为 $(\mathbf{A}, \mathbf{B})$, 其中 $\mathbf{A} \in \mathbb{Z}^{m \times m}$, $\mathbf{B} \in \mathbb{Z}^{m \times N}$. 设 $\mathbf{b}$ 为矩阵 $\mathbf{B}$ 的第1列, $m = (k_1 + k_2)N$.

5.3.1 Primal Attack

构造格

$$\mathcal{L}_q(\mathbf{A}, \mathbf{I}_m, -\mathbf{b}) = \{\mathbf{x} \in \mathbb{Z}^{2m+1} : (\mathbf{A}, \mathbf{I}_m, -\mathbf{b})\mathbf{x} \bmod q = \mathbf{0}\}.$$

格的维数为 $d = 2m + 1$, 体积为 q^m . 容易验证 $(\mathbf{s}, \mathbf{e}, 1)$ 为格短向量, 于是我们可用BKZ算法来寻找这个短向量. BKZ算法成功找到这个短向量的条件为 $\sigma\sqrt{\beta} < \delta_\beta^{2\beta-d-1} \cdot q^{m/d}$, 其中 σ 为噪声向量的标准差.

在实际攻击过程中, 因为向量 $\mathbf{s}$ 和 $\mathbf{e}$ 的元素比较小, 所以我们可以猜测部分 $\mathbf{s}$ 或者 $\mathbf{e}$ 的元素的值, 然后再应用BKZ算法, 这样可以稍微减少格的维数, 从而降低BKZ算法的复杂度. 但是, 我们不能猜测太多 $\mathbf{s}$ 或者 $\mathbf{e}$ 的元素的值, 否则猜测的复杂度会很高. 我们需要平衡猜测的元素个数和BKZ算法的复杂度. 这就是所谓的Hybrid Primal Attack.

5.3.2 Dual Attack

我们首先构造对偶格

$$\mathcal{L}_q(\mathbf{A}) = \{(\mathbf{v}, \mathbf{w}) \in \mathbb{Z}^{2m} : \mathbf{w}^\top = \mathbf{v}^\top \mathbf{A} \bmod q\},$$

格的维数为 $d = 2m$, 体积为 q^m . 利用BKZ算法, 可以找到长度约为 $\ell = \delta_\beta^{d-1} q^{1/2}$ 的短向量 $(\mathbf{v}, \mathbf{w})$. 我们利用这个短向量来构造区分器, 以区分LWE样本和随机样本. 给定向量 $\mathbf{b} \in \mathbb{Z}^m$, 计算 $z \equiv \mathbf{v}^\top \mathbf{b} \equiv \mathbf{v}^\top \mathbf{A} \mathbf{s} + \mathbf{v}^\top \mathbf{e} \equiv \mathbf{w}^\top \mathbf{s} + \mathbf{v}^\top \mathbf{e} \pmod{q}$. 如果 $\mathbf{b} = \mathbf{A} \mathbf{s} + \mathbf{e}$, 那么 z 服从标准差为 $\ell\sigma$ 的高斯分布. 如果 $\mathbf{b}$ 服从均匀分布, 那么 z 也服从均匀分布. 高斯分布和均匀分布的统计距离为 $\epsilon = 4 \exp(-2\pi^2 \tau^2)$, 其中 $\tau = \ell\sigma/q$. 所以, 我们需要用BKZ算法生成 $1/\epsilon^2$ 个短向量. BKZ的筛算法生成 $2^{0.2075\beta}$ 个短向量, 假设这些短向量的长

度都不超过 ℓ , 我们需要执行 ϵ -区分器 $R = \max\{1, 1/2^{0.2075\beta\epsilon^2}\}$ 次, 才能最终恢复密钥(或者等价密钥) $\mathbf{s}$.

与Primal攻击相似, 在实际攻击过程中, 因为向量 $\mathbf{s}$ 和 $\mathbf{e}$ 的元素比较小, 所以我们可以猜测部分 $\mathbf{s}$ 或者 $\mathbf{e}$ 的元素的值, 然后再应用BKZ算法, 以降低攻击的复杂度.

5.3.3 Loong密钥封装算法的攻击复杂度

表4给出了Primal攻击和dual攻击LoonKEM的计算复杂度.

Algorithm	Primal Attack		Dual Attack	
	Classical	Quantum	Classical	Quantum
Loong128	149	136	147	134
Loong256	295	269	292	264
Loong384	407	375	398	361
Loong512	535	496	521	478

表 4: Primal攻击和dual攻击LoonKEM的计算复杂度, 表中的数据为攻击复杂度的以2为底的对数值.

6 优势和劣势

Loong密钥封装算法易于实现和扩展. 不同的安全强度的Loong密钥封装算法的实现只有头文件的宏定义不同, 其他代码可以复用. 如果需要调整安全强度, 只需要更改头文件的宏定义.

下面我们通过与其他后量子密钥封装方案的比较来说明Loong密钥封装算法的优势和劣势.

与直接基于LWE问题的方案相比 直接用LWE问题来构造密钥封装算法或密钥交换协议的方法主要有两种. 第一种方法详见文献[31], 这种方法的公钥和私钥都很大, 密文相对小. 另一种相对实用的方法详见文献[13], 即FrodoKEM密钥封装算法, 从表5 可见, FrodoKEM-640的公钥和密文长度大约为Loong密钥封装算法的6倍. 在安全方面, 直接基于LWE问题的密钥封装算法没有代数结构, 而Loong密钥封装算法有一些代数结构.

Scheme	sk	pk	ct	gen	enc	dec
Loong128	1832	1472	1512	2621	2647	2676
FrodoKEM-640[13]	19888	9616	9720	1385	1862	1747
Kyber512[26]	1632	800	768	123	155	188
NewHope512[27]	1888	928	1120	117	180	206
ntruhs2048509[28]	935	699	699	8906	643	1662
hqc-128[29]	40	2249	4497	180	408	875
mceliece348864[30]	6492	261120	128	58034	44	135

表 5: 128比特安全的密钥封装算法比较, 其中**sk**、**pk**和**ct**的大小的单位为Bytes; **gen**、**enc** 和**dec**的单位为 10^3 Cycles; 它们均为参考实现的性能.

与基于MLWE问题的方案相比 典型的基于MLWE问题的密钥封装算法为Kyber密钥封装算法[26], 它是美国的后量子密钥封装标准算法. 在安全方面, Kyber512引入的代数结构为 $Z_q[X]/X^{256} + 1$, 而Loong128引入的代数结构为 $Z_q[X]/X^{12} + 1$. Kyber512的代数结构较强, 而Loong128的代数结构相对较弱, 且Loong128的公钥矩阵有一部分没有代数结构. 在性能方面, 从表5可见, Kyber512密钥封装算法明显优于Loong128密钥封装算法. Loong密钥封装方案是一个FrodoKEM密钥封装方案和Kyber密钥封装方案折中方案.

与基于RLWE问题的方案相比 典型的基于RLWE问题的密钥封装算法为NewHope密钥封装算法[27]. 在安全方面, NewHope512密钥封装算法引入的代数结构为 $Z_q[X]/X^{512} + 1$, 此代数结构比Loong128引入的代数结构强. 在性能方面, 从表5可见, NewHope512密钥封装算法明显优于Loong128密钥封装算法.

与NTRU相比 在文献[28]中, ntruhs2048509密钥封装方案的代数结构为 $Z_q[X]/X^{509} - 1, q = 2048$. 此代数结构比Loong128引入的代数结构强. 在性能方面, 从表5可见, ntruhs2048509密钥封装算法明显优于Loong128密钥封装算法.

与基于编码问题的方案相比 典型的基于编码问题的密钥封装算法为Classic McEliece[30]密钥封装算法和HQC[29]密钥封装算法. 从表5可

见, 此两个算法的公钥和密文加起来的总长度比Loong128长, 公钥和私钥生成速度比Loong128慢, 密钥封装和解封装速度比Loong128快.

综上所述, 在实际应用中, 基于RLWE问题的密码方案、基于MLWE问题的密码方案和基于NTRU问题的密码方案都具有很强的代数结构. 直接基于LWE问题的密码方案的公钥和密文的长度太长. 如果你对具有强代数结构的密码方案的安全性有疑虑, 又不希望公钥和密文长度太长, 那么Loong密钥封装算法是一个很好的折中方案.

参考文献

- [1] Miklós Ajtai. Generating hard instances of lattice problems. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pages 99–108, 1996.
- [2] Jeffrey Hoffstein, Jill Pipher, and Joseph H Silverman. Ntru: A ring-based public key cryptosystem. In *International algorithmic number theory symposium*, pages 267–288. Springer, 1998.
- [3] Miklós Ajtai and Cynthia Dwork. A public-key cryptosystem with worst-case/average-case equivalence. In *Proceedings of the twenty-ninth annual ACM symposium on Theory of computing*, pages 284–293, 1997.
- [4] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In *Symposium on the Theory of Computing*, 2005.
- [5] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 1–23. Springer, 2010.
- [6] Jintai Ding, Xiang Xie, and Xiaodong Lin. A simple provably secure key exchange scheme based on the learning with errors problem. *Cryptology ePrint Archive*, 2012.
- [7] Chris Peikert. Lattice cryptography for the internet. In *International workshop on post-quantum cryptography*, pages 197–219. Springer, 2014.

- [8] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange-a new hope. *Cryptology ePrint Archive*, 2015.
- [9] Adeline Langlois and Damien Stehlé. Worst-case to average-case reductions for module lattices. *Designs, Codes and Cryptography*, 75(3):565–599, 2015.
- [10] Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-kyber: a cca-secure module-lattice-based kem. In *2018 IEEE European symposium on security and privacy (EuroS&P)*, pages 353–367. IEEE, 2018.
- [11] Deepraj Soni, Kanad Basu, Mohammed Nabeel, Najwa Aaraj, Marc Manzano, Ramesh Karri, Deepraj Soni, Kanad Basu, Mohammed Nabeel, Najwa Aaraj, et al. Crystals-dilithium. *Hardware Architectures for Post-Quantum Digital Signature Schemes*, pages 13–30, 2021.
- [12] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In *Annual international cryptology conference*, pages 537–554. Springer, 1999.
- [13] Lewis Glabush, Patrick Longa, Michael Naehrig, Chris Peikert, Douglas Stebila, and Fernando Virdia. Frodokem: A cca-secure learning with errors key encapsulation mechanism. *Cryptology ePrint Archive*, 2025.
- [14] Jan-Pieter D’Anvers, Angshuman Karmakar, Sujoy Sinha Roy, and Frederik Vercauteren. Saber: Module-lwr based key exchange, cpa-secure encryption and cca-secure kem. In *International Conference on Cryptology in Africa*, pages 282–305. Springer, 2018.
- [15] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the fujisaki-okamoto transformation. In Yael Kalai and Leonid Reyzin, editors, *Theory of Cryptography*, pages 341–371, Cham, 2017. Springer International Publishing.

- [16] Thijs Laarhoven. Sieving for shortest vectors in lattices using angular locality-sensitive hashing. In *Annual Cryptology Conference*, pages 3–22. Springer, 2015.
- [17] Anja Becker and Thijs Laarhoven. Efficient (ideal) lattice sieving using cross-polytope lsh. In *International Conference on Cryptology in Africa*, pages 3–23. Springer, 2016.
- [18] Anja Becker, Léo Ducas, Nicolas Gama, and Thijs Laarhoven. New directions in nearest neighbor searching with applications to lattice sieving. In *Proceedings of the twenty-seventh annual ACM-SIAM symposium on Discrete algorithms*, pages 10–24. SIAM, 2016.
- [19] Martin R Albrecht, Léo Ducas, Gottfried Herold, Elena Kirshanova, Eamonn W Postlethwaite, and Marc Stevens. The general sieve kernel and new records in lattice reduction. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 717–746. Springer, 2019.
- [20] Thijs Laarhoven, Michele Mosca, and Van De Pol Joop. Finding shortest lattice vectors faster using quantum search. *Designs, Codes and Cryptography*, 77(2):375–400, 2015.
- [21] André Chailloux and Johanna Loyer. Lattice sieving via quantum random walks. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 63–91, 2021.
- [22] Max Heiser. Improved quantum hypercone locality sensitive filtering in lattice sieving. *Cryptology ePrint Archive*, 2021.
- [23] Divesh Aggarwal, Jianwei Li, Phong Q Nguyen, and Noah Stephens-Davidowitz. Slide reduction, revisited-filling the gaps in svp approximation. In *Annual International Cryptology Conference*, pages 274–295. Springer, 2020.
- [24] Tsuyoshi Takagi and Thomas Peyrin. Revisiting the expected cost of solving usvp and applications to lwe. *Springer, Cham*, 10.1007/978-3-319-70694-8(Chapter 11):297–322, 2017.

- [25] Yuanmi Chen and Phong Q. Nguyen. Bkz 2.0: Better lattice security estimates. *DBLP*, 2011.
- [26] Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-kyber algorithm specifications and supporting documentation. 2021.
- [27] Erdem Alkim, Roberto Avanzi, Joppe Bos, Léo Ducas, Antonio De La Piedra, Peter Schwabe Thomas Pöppelmann, and Douglas Stebila. Newhope. *Submission to round, 1*, 2017.
- [28] Cong Chen, Oussama Danba, Jeffrey Hoffstein, Andreas Hülsing, Joost Rijneveld, John M Schanck, Peter Schwabe, William Whyte, and Zhenfei Zhang. Algorithm specifications and supporting documentation. *Brown University and Onboard security company, Wilmington USA*, 2019.
- [29] Philippe Gaborit, Carlos Aguilar-Melchor, Nicolas Aragon, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Edoardo Persichetti, Gilles Zémor, Jurjen Bos, et al. Hamming quasi-cyclic (hqc). *NIST Post-Quantum Standardization, 4th Round; National Institute of Standards and Technology: Gaithersburg, MD, USA*, 2025.
- [30] Daniel J Bernstein, Tung Chou, Tanja Lange, Ingo von Maurich, Rafael Misoczki, Ruben Niederhagen, Edoardo Persichetti, Christiane Peters, Peter Schwabe, Nicolas Sendrier, et al. Classic mceliece: conservative code-based cryptography. *NIST submissions*, 1(1):1–25, 2017.
- [31] Chris Peikert, Vinod Vaikuntanathan, and Brent Waters. A framework for efficient and composable oblivious transfer. In *Annual international cryptology conference*, pages 554–571. Springer, 2008.